aSERG / UFMG
APPLIED SOFTWARE ENGINEERING RESEARCH GROUP

PPGCC
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO DCC-UFMG

Identifying Key Developers in Software Projects using Code Authorship Metrics

Guilherme Avelino
(PhD candidate)

Marco Tulio Valente
(Advisor)

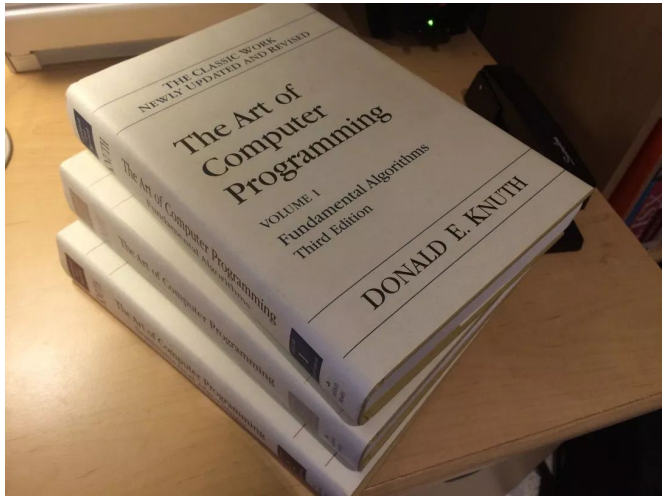
Thesis defense, UFMG, 2018

UFPI 1971 UNIVERSIDADE FEDERAL DO PIAUÍ

Authorship is precisely documented in most intellectual work

Authorship

Books





Authorship

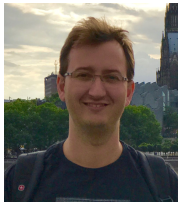
Music





Authorship

Scientific Papers









Who Can Maintain this Code?

Assessing the Effectiveness of Repository-Mining Techniques for Identifying Software Maintainers

Guilherme Avelino[1,2], Leonardo Passos[3], Fabio Petrillo[4], Marco Tulio Valente[1]

[1]Federal University of Minas Gerais, Brazil
[2]Federal University of Piaui, Brazil
[3]Quantstamp Technologies, Canada
[4]Polytechnique Montreal, Canada
{gaa, mtov}@dcc.ufmg.br, leo@quantstamp.com, fabio@petrillo.com

Authorship

Source code?

Author names are not stamped in the code

Authorship evolve with time

Modified by many developers

```
if (tf.getTf()>0) {
  if (nLeavers == tf.getTf()) {
    if (equalSets(lastTFEventDevelopers,
        tfDevelopers) ||
        (lastTFEventMeasure!=null && !measu
      measure.setTFEvent(false);
      measure.setSurviveEvent(false);
    } else {
      measure.setTFEvent(true);
      measure.setSurviveEvent(false);
      lastTFEventDevelopers = tfDevelopers;
      lastTFEventMeasure = measure;
    }
```



Studies on Code Authorship

- Recommend code experts [*McDonald, 2000; Mockus, 2002; Minto, 2007*]
- Notify developers on changes of interest [*Hattori, 2007; Ma, 2009*]
- Relationship with quality [*Bird, 2011; Greiler, 2015, Thongtanunam, 2016*]

Studies on Code Authorship

- Recommend code experts [*McDonald, 2000; Mockus, 2002; Minto, 2007*]
- Notify developers on changes of interest [*Hattori, 2007; Ma, 2009*]
- Relationship with quality [*Bird, 2011; Greiler, 2015, Thongtanunam, 2016*]

Lack of studies that **rely on code authorship** to investigate the **organization of software teams**

Thesis Goal

- Based on code authorship measures:
  - propose a methodology to **identify key developers** of a system
  - using this methodology, study **how development teams are organized**
  - and also investigate **practical applications** of code authorship on **Software Engineering problems**

Thesis Roadmap

1. Methodology for Authors Identification
2. Study: Analyzing Code Authorship
3. Application 1: key developers in a project level
   - Estimating Truck Factors
   - Investigating Truck Factor Events
4. Application 2: key developers in a file level
   - Identifying Software Maintainers

Author Identification



Code Authorship Metric

- Degree-of-authorship (DOA)
  - First Authorship (FA)
  - Number of Deliveries (DL)
  - Number of Acceptances (AC)

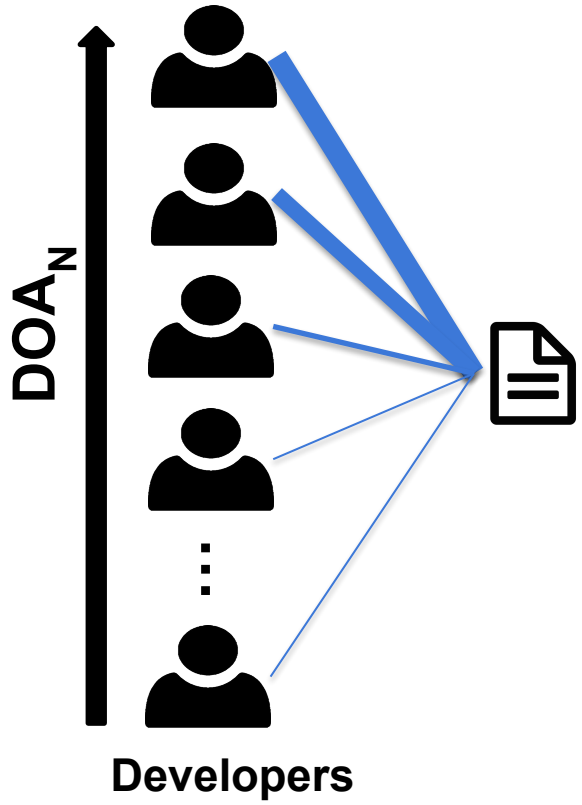
$$DOA(d, f) = 3.293 + 1.098 * FA + 0.164 * DL - 0.321 * ln(1 + AC)$$

Fritz, T., et al. Degree-of-knowledge: modeling a developer's knowledge of code. ACM TOSEM 2014.

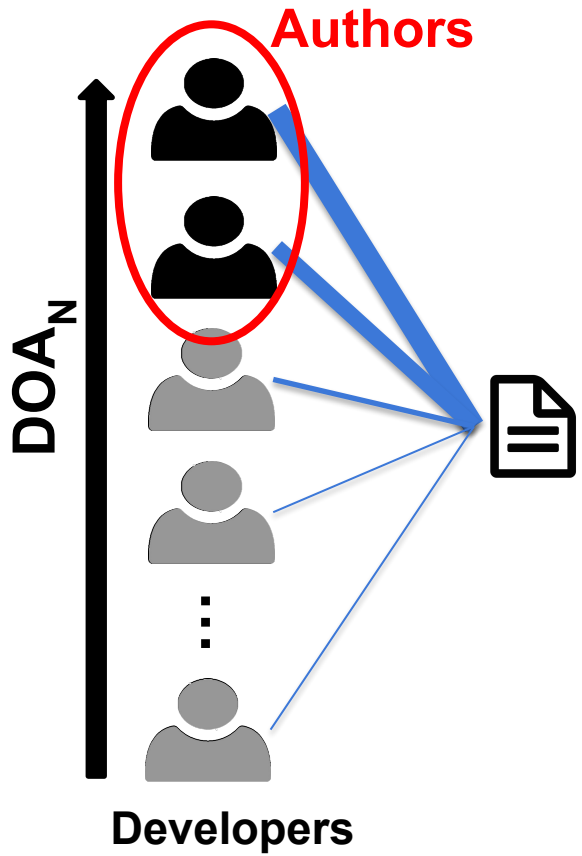
Normalized DOA (DOA_N)

- Suppose a file f:
  - DOA (Joao, f) = 5
  - DOA(Maria, f) = 4
  - DOA(Jose, f) = 2
- We use normalized DOA values:
  - DOA_N(Joao, f) = 5 / 5 = 1
  - DOA_N(Maria, f) = 4 / 5 = 0.8
  - DOA_N(Jose, f) = 2 / 5 = 0.4

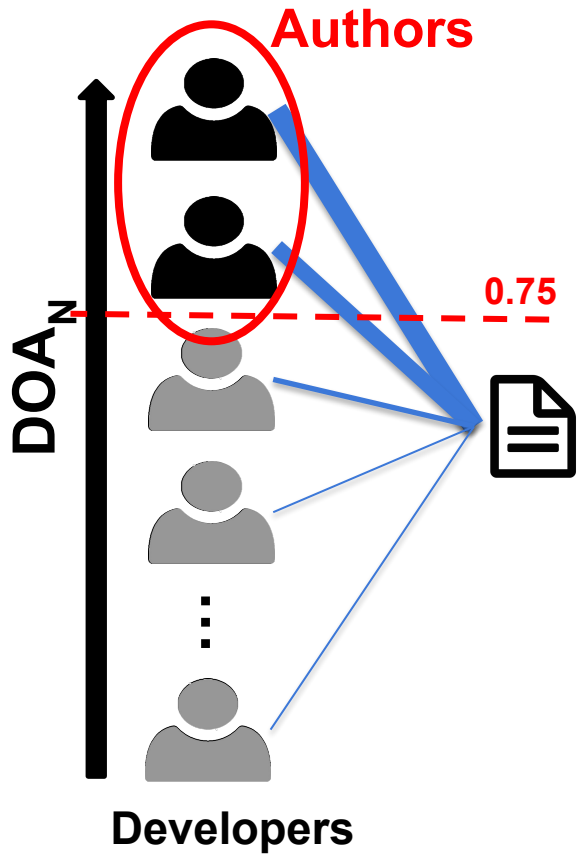
Author Identification



Author Identification

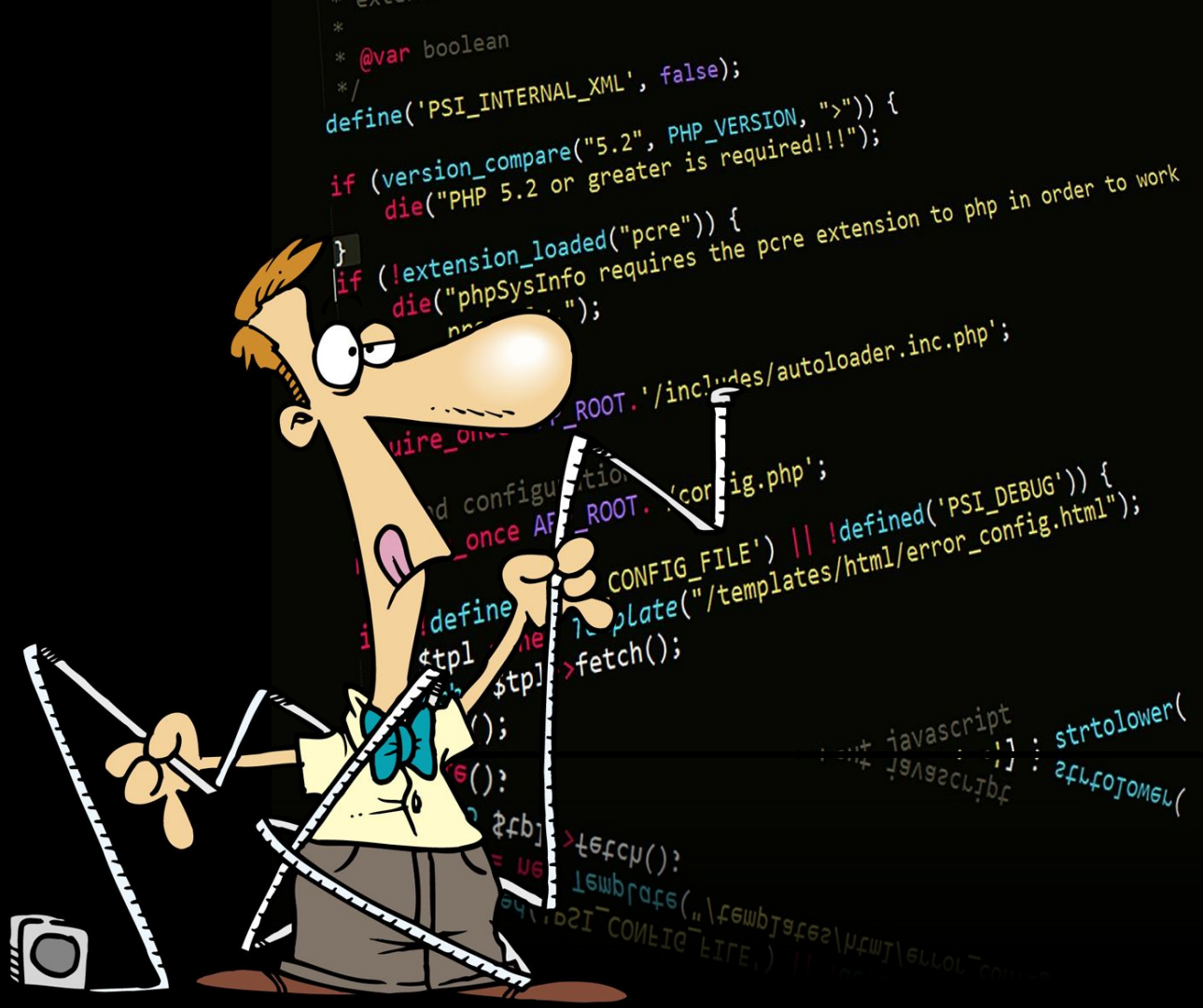


Author Identification



Empirically defined:
6 systems of different languages

Analyzing Code Authorship



Goals

- Use **code authorship measures** to investigate the **organization of development teams**
- We propose a set of authorship-centric concepts
  - 1st Study - Linux kernel evolution
  - 2nd Study - 114 popular GitHub systems

Research Questions

RQ1: What is the proportion of authors/developers?

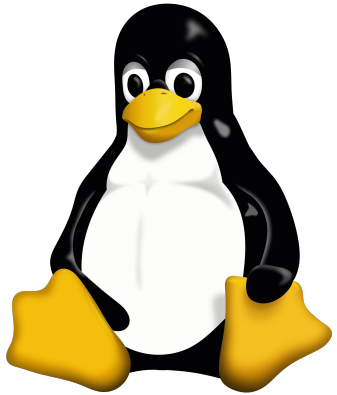
RQ2: What is the distribution of files per author?

RQ3: How specialized is the work?

RQ4: What are the properties of the co-authorship network?

1st Study

Linux Kernel



Linux Kernel

- GitHub repository
  - 56 stable releases (11 years)
- Architectural Decomposition
  - Arch, Core, Driver, FS, Net and Misc

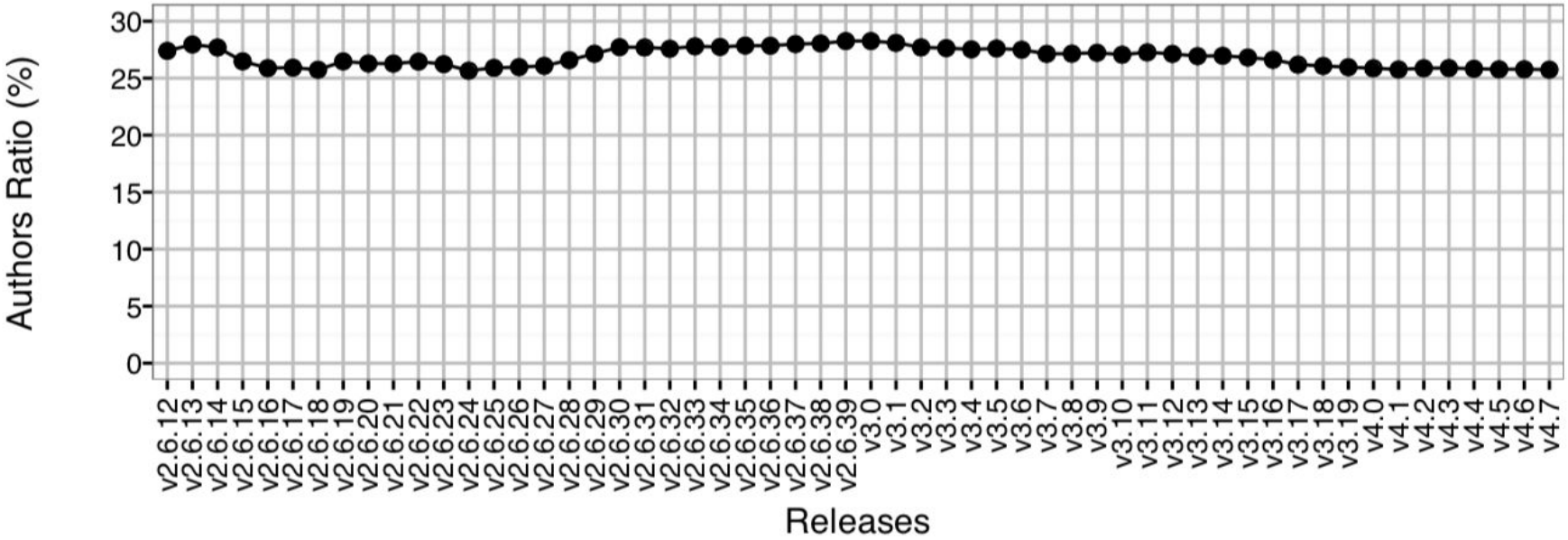
RQ1. Proportion of Authors

Linux (ver. 4.7) has 13k developers,
but **only 26% are authors**



RQ1. Proportion of Authors

Mean 26.86 ± 0.83%



RQ2. Files per Author

- Few developers authored many files

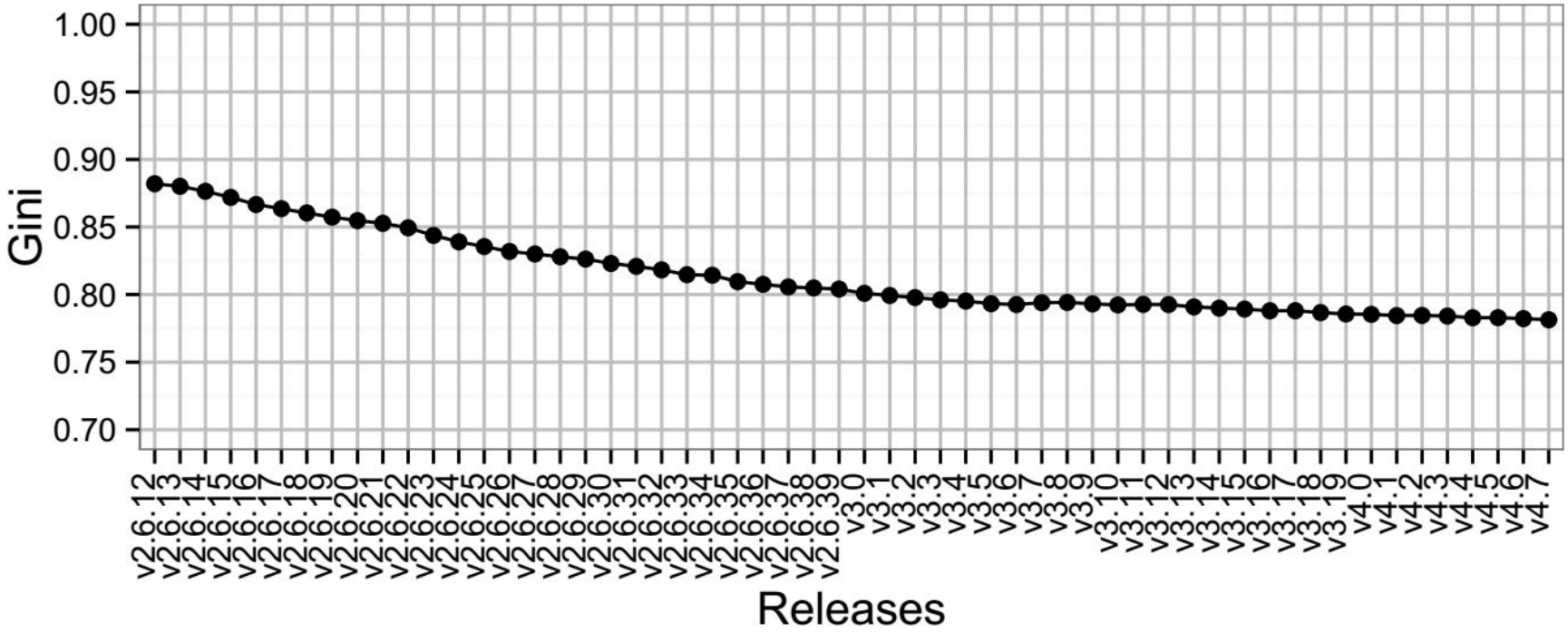
3% = more than 100 files

- Many developers authored few files

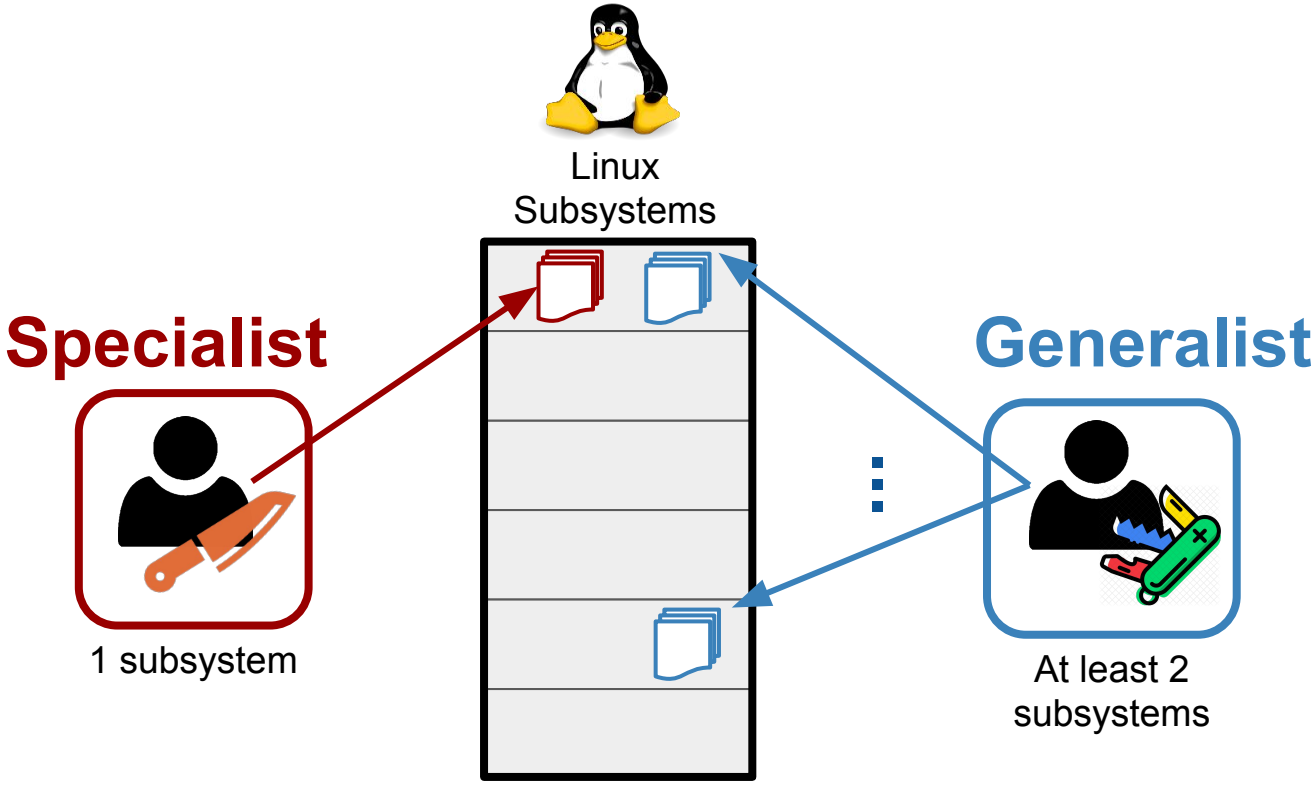
75% = max of 11 files



RQ2. Files per Author (Gini)



RQ3. Work Specialization



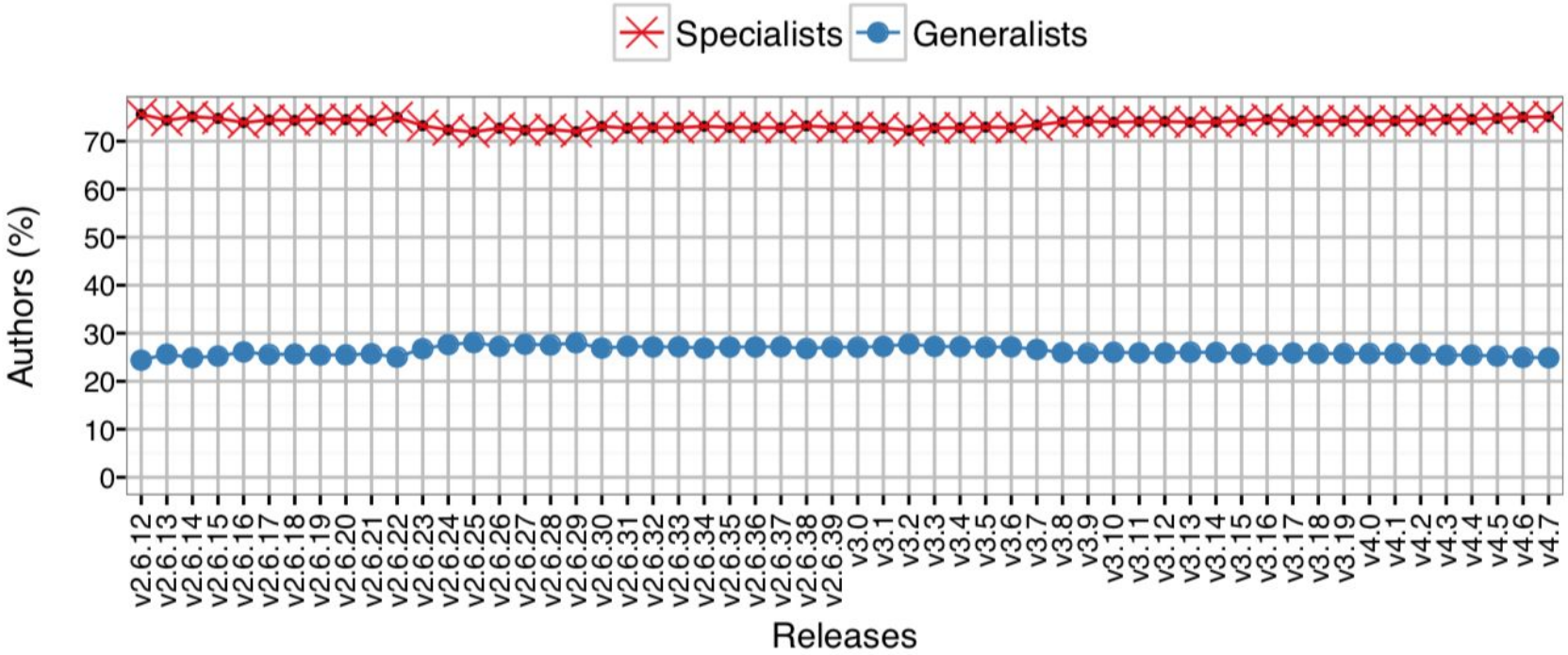
RQ3. Work Specialization



Specialists ≥ 72%

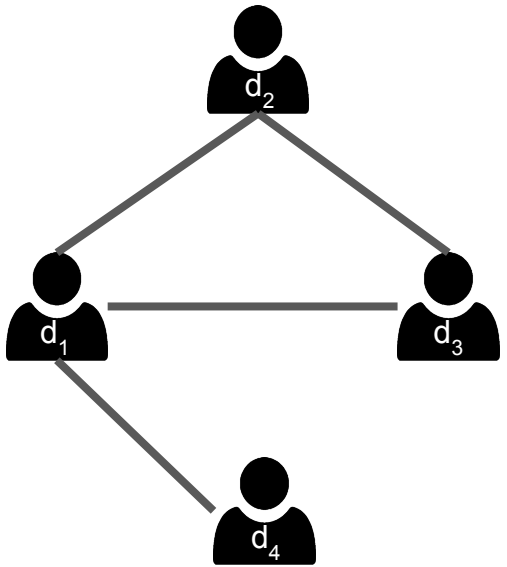


Generalists < 28%



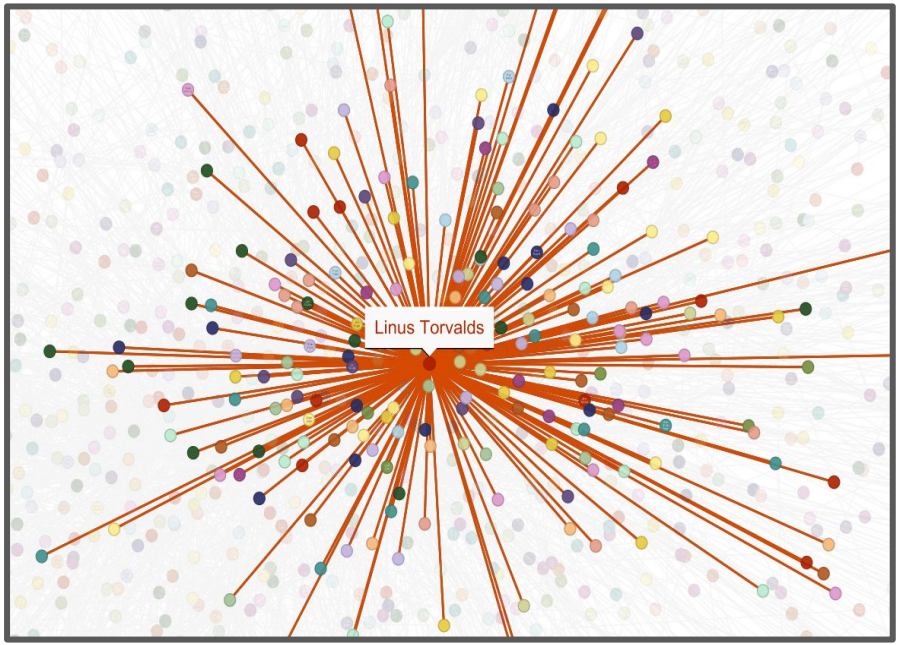
RQ4. Co-authorship Properties

- Co-authorship network
  - Nodes are authors
  - Edges connect co-authors in at least one file
  - Examples:
    - authors(f_1) = {d_1, d_2, d_3}
    - authors(f_2) = {d_1, d_4}



RQ4. Co-authorship Properties

- Linux kernel
  - **Mean degree = 3.44**



Torvalds degree = 215

2nd Study

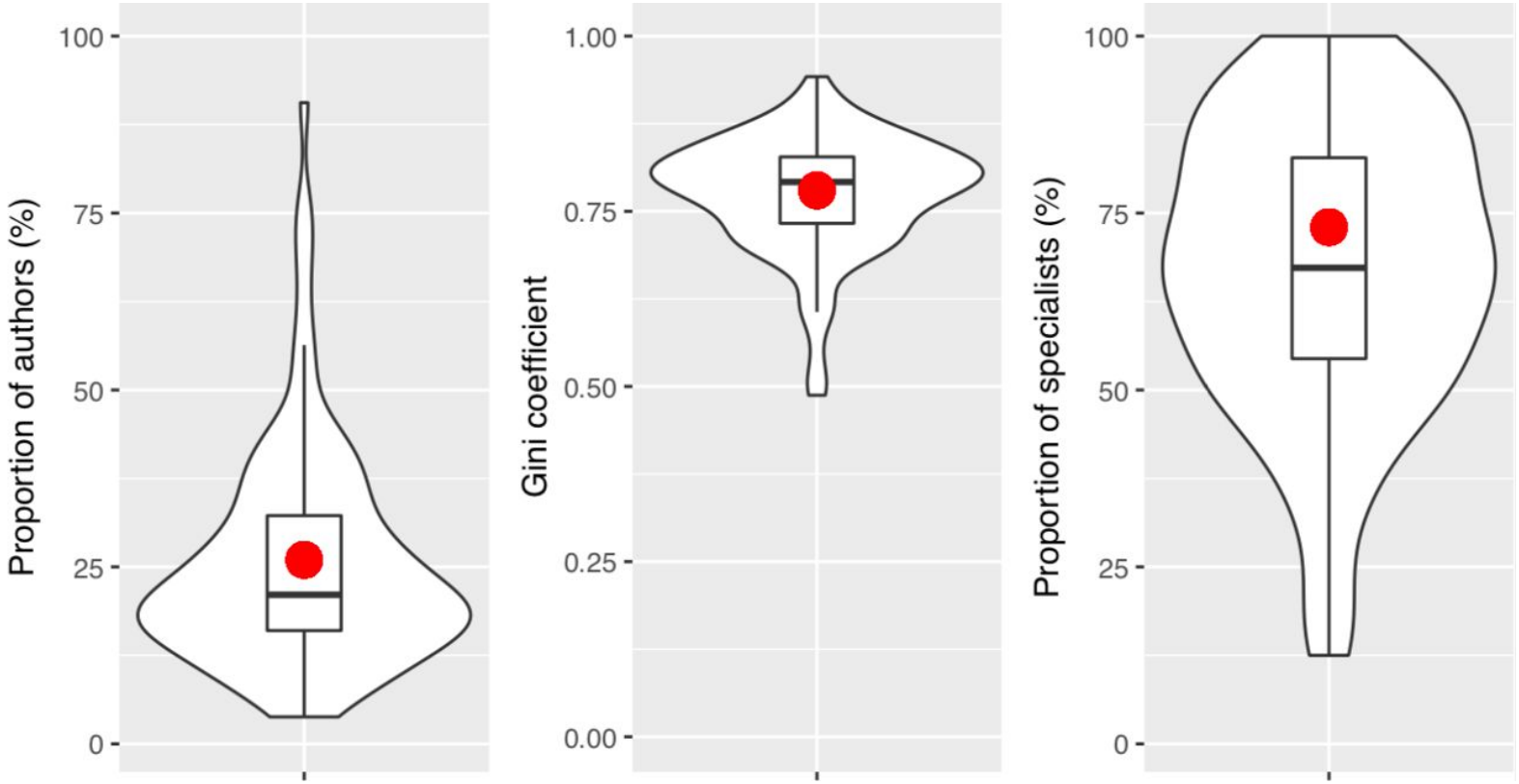
114 GitHub projects



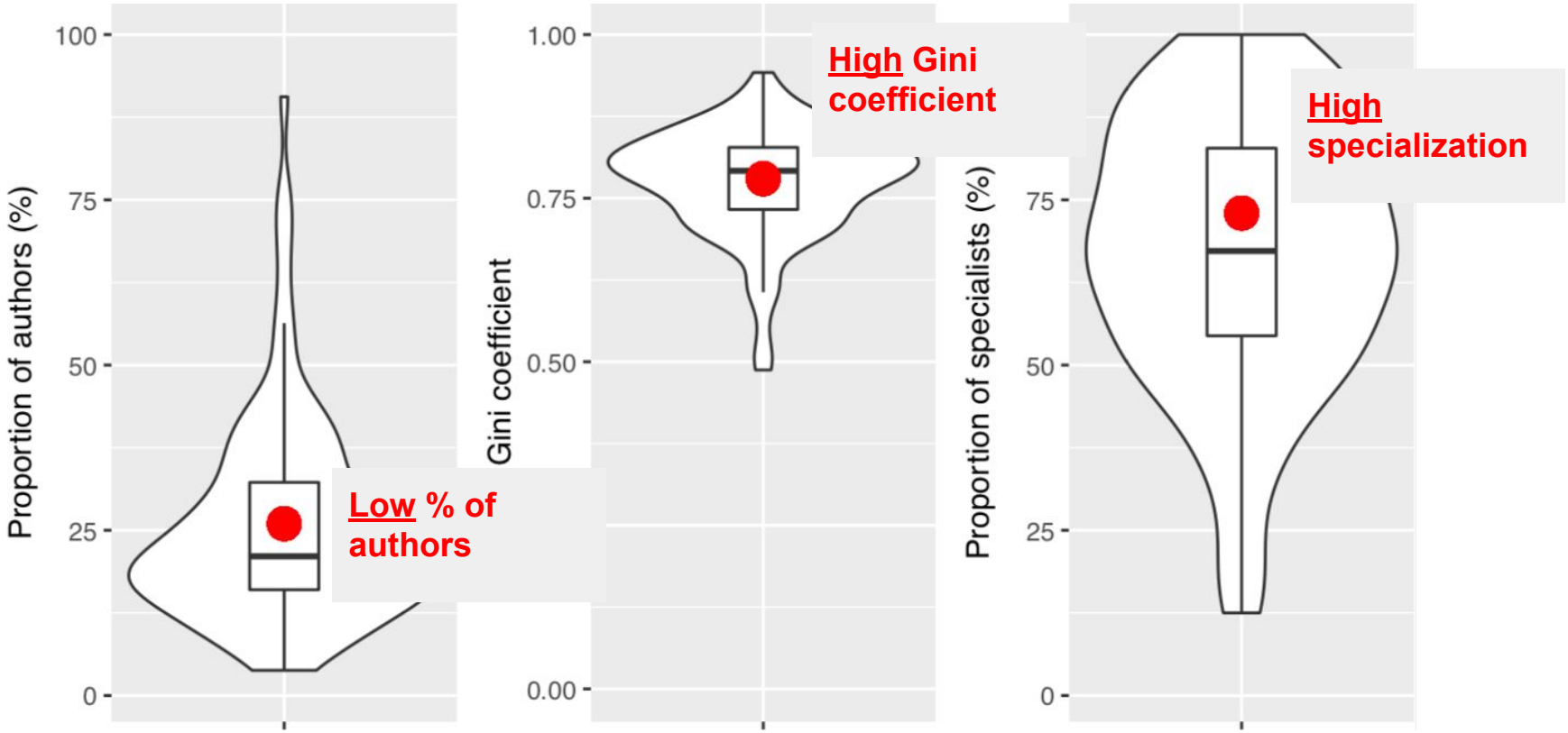
Dataset

- 114 GitHub projects
  - 6 programming languages
  - Popular projects (by number of stars)

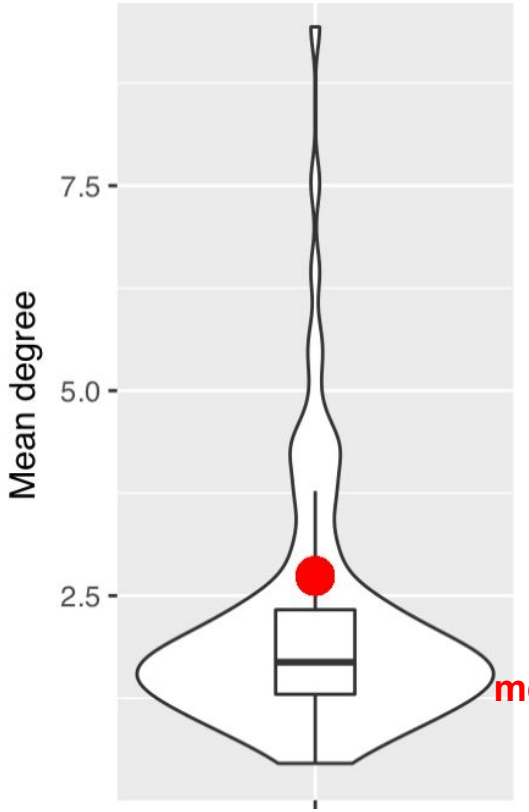
Authorship Measures



Authorship Measures

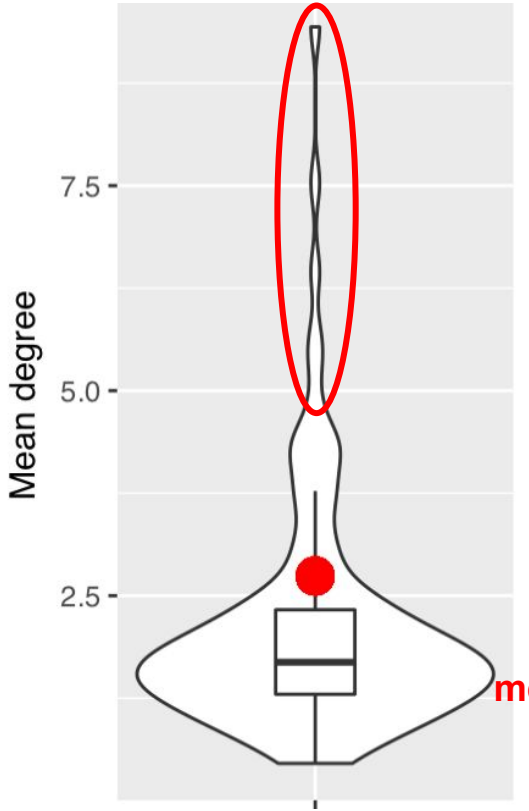


Co-authorship Measures



Low mean degree

Co-authorship Measures



IntelliJ IDEA **9.44**

7.53

4.43

2.74

Low mean degree

Contributions

1. A **methodology and a set of concepts** to study the organization of software teams

2. **Results about authorship** in 1+114 GitHub projects
   - Low proportion of authors
   - Skewed distribution of files per author
   - High proportion of specialists

Estimating Truck Factors



“What if you saw this posted tomorrow:

Guido’s unexpected death has come as a shock to us all. Disgruntled members of the Tcl mob are suspected, but no smoking gun has been found...”

Python’s mailing list discussion, 1994.

Truck Factor

"The number of people on your team that have to be hit by a truck (or quit) before the project is in serious trouble"

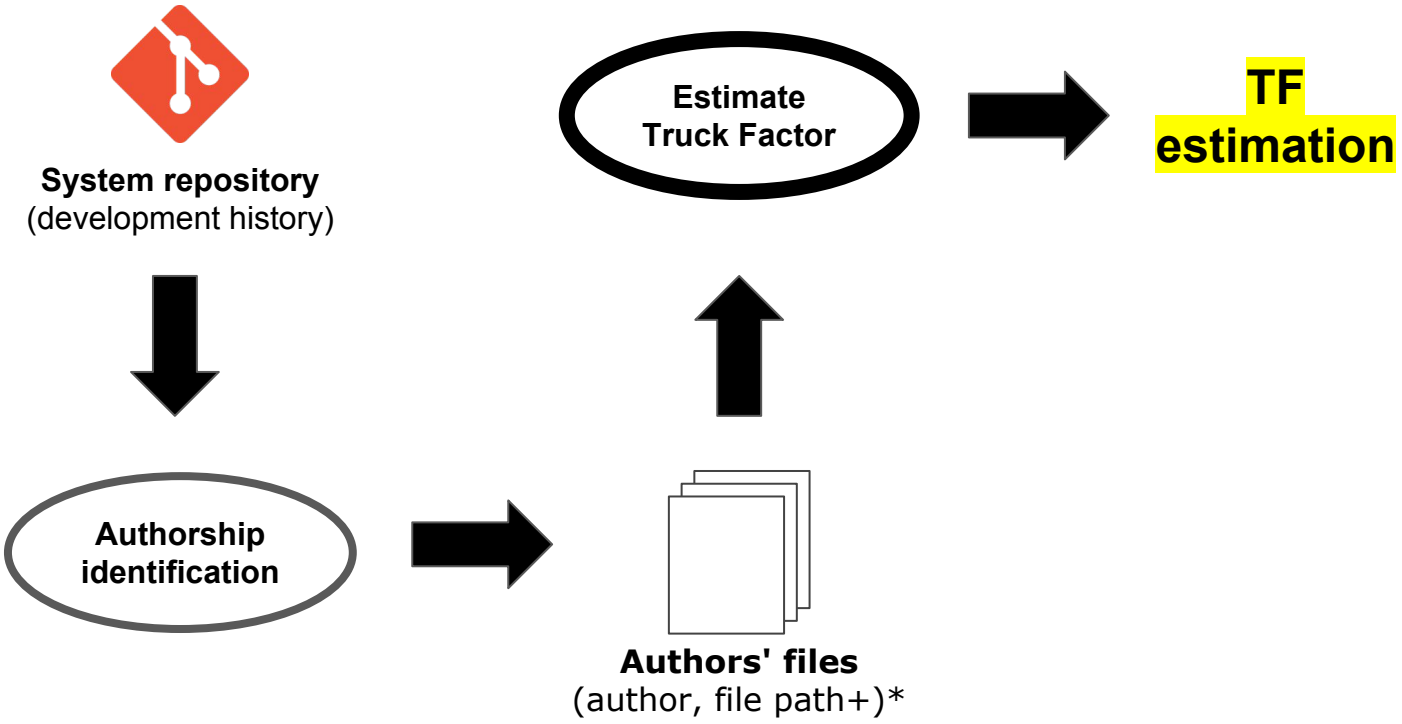


Existing Approaches

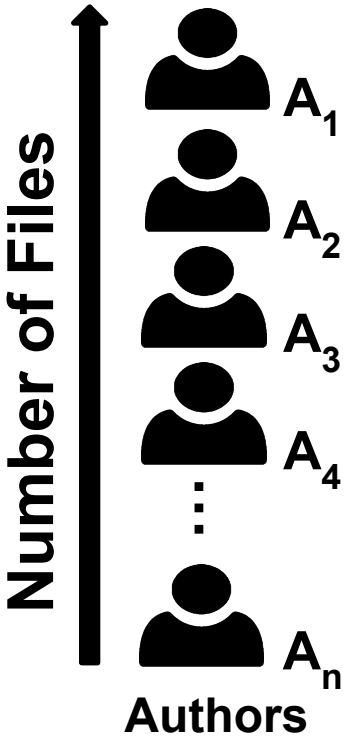
- Only for small teams (< 30 developers)
- No empirical evidence from real-world systems

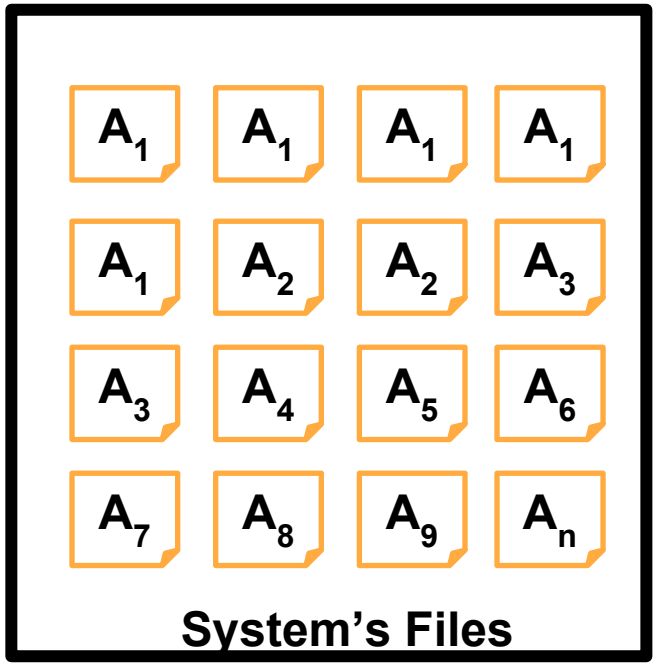
Zazworka et al. (ESEM 2010) and Ricca et al. (ESEM 2010)

Proposed Approach

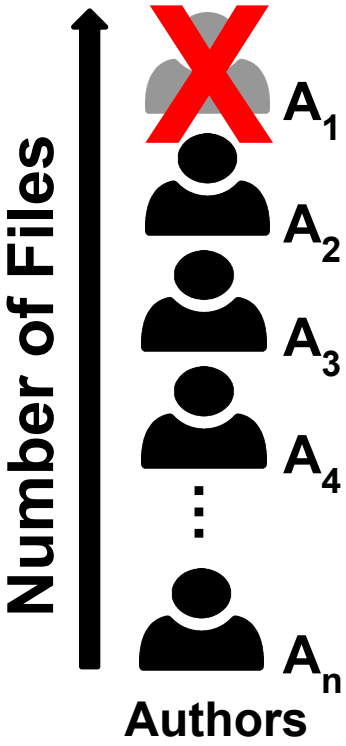


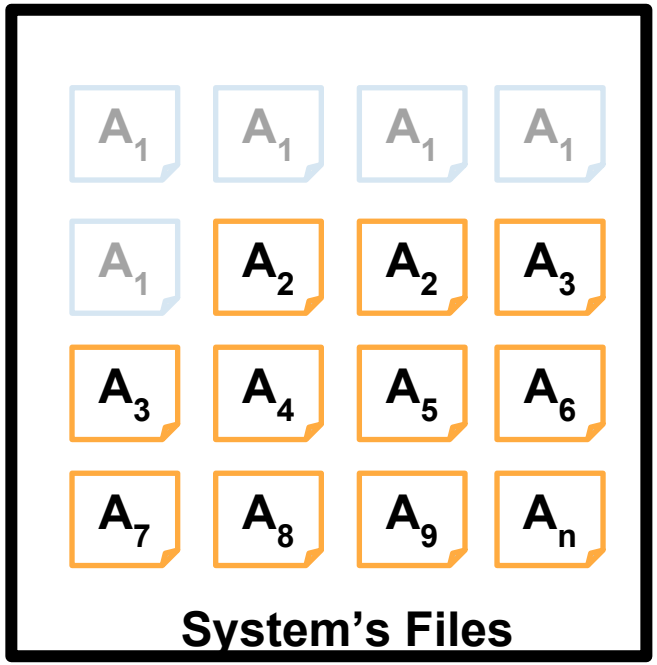
Estimating Truck Factors



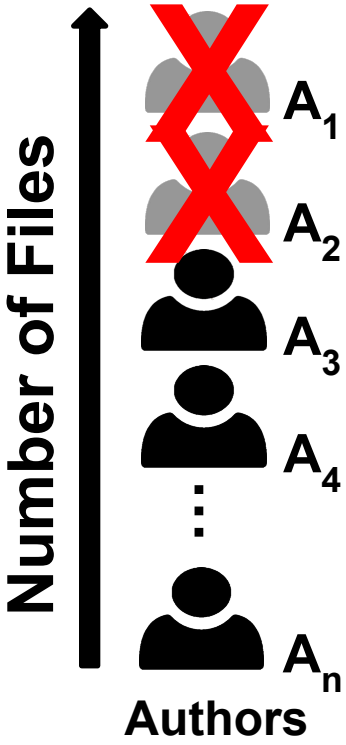


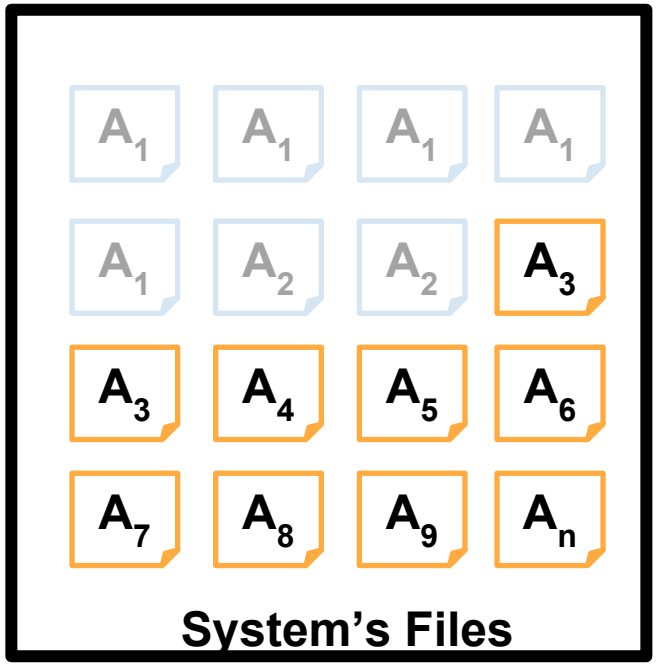
Estimating Truck Factors



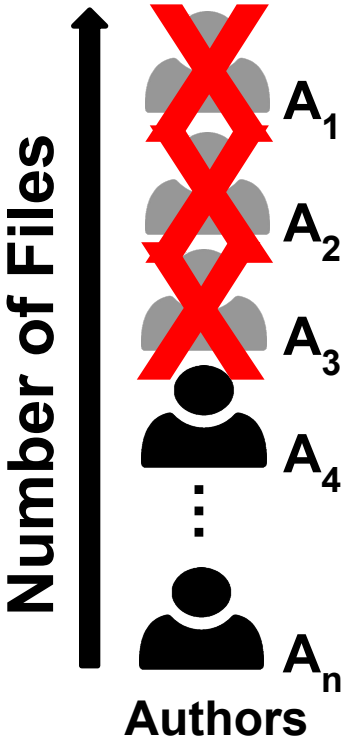


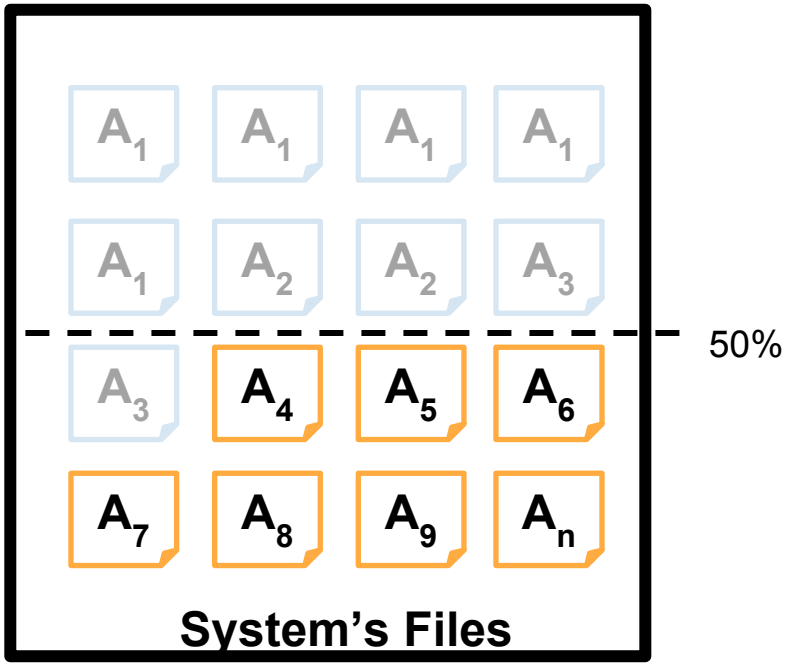
Estimating Truck Factors



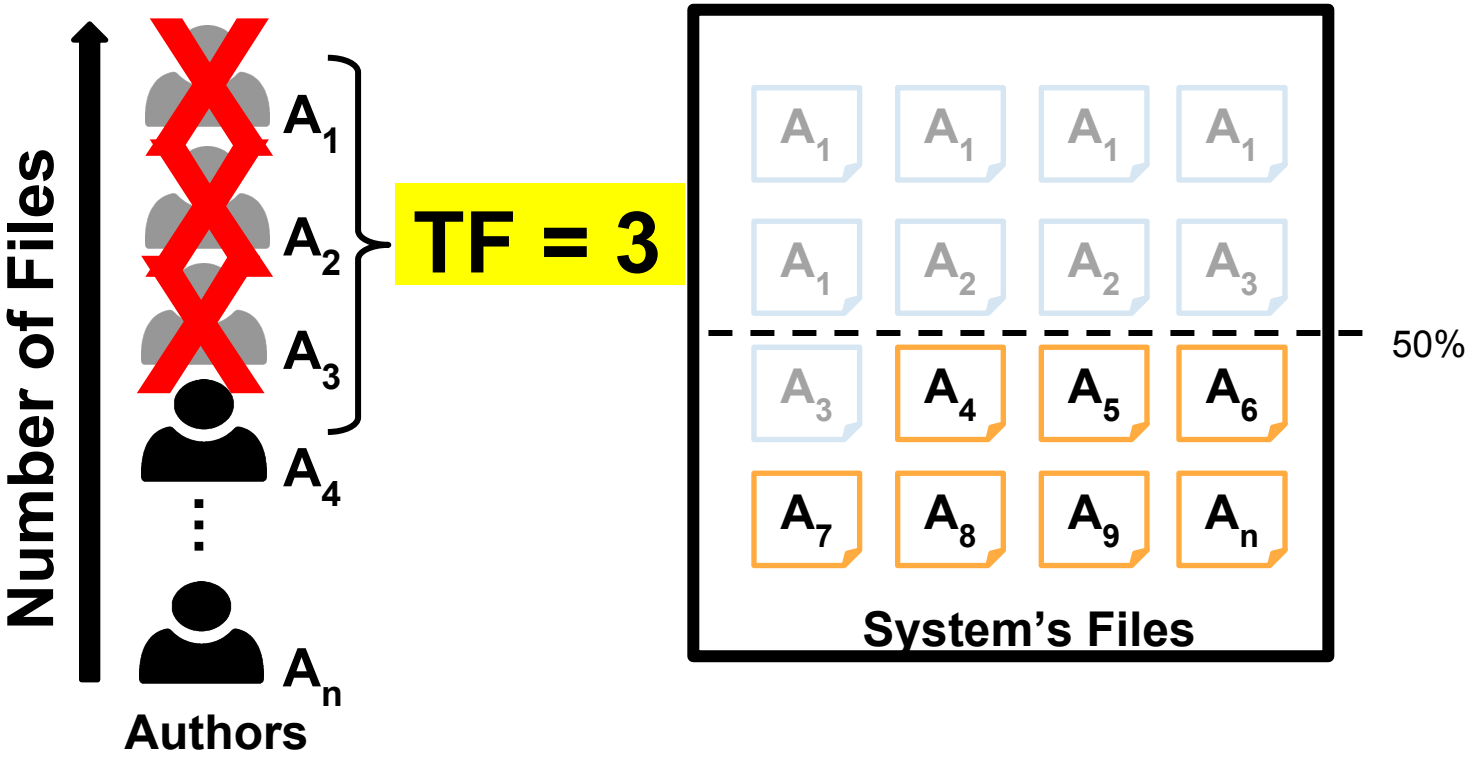


Estimating Truck Factors





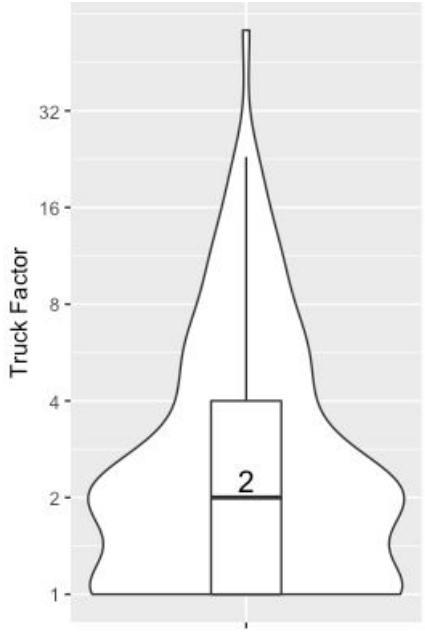
Estimating Truck Factors



Truck Factor Results

133 projects





(TF ≤ 2) 65%

Survey

- GitHub issues
  - Opened: 114
  - Answered: 62

Response ratio: 54%

Do developers agree that the top-ranked authors are the main developers of their projects?

84% agree/partially agree

Do developers agree that their projects will be in trouble if they lose the truck factor authors?

53% agree/partially agree

Contributions

1. **Novel approach** to estimate truck factors

2. **Truck factors** of 133 systems
   - Most of them have low TFs

Practical Impact

10K downloads

PeerJ PrePrints

What is the Truck Factor of Popular GitHub Applications? A First Assessment

Guilherme Avelino, Marco Tulio Valente, Andre Hora

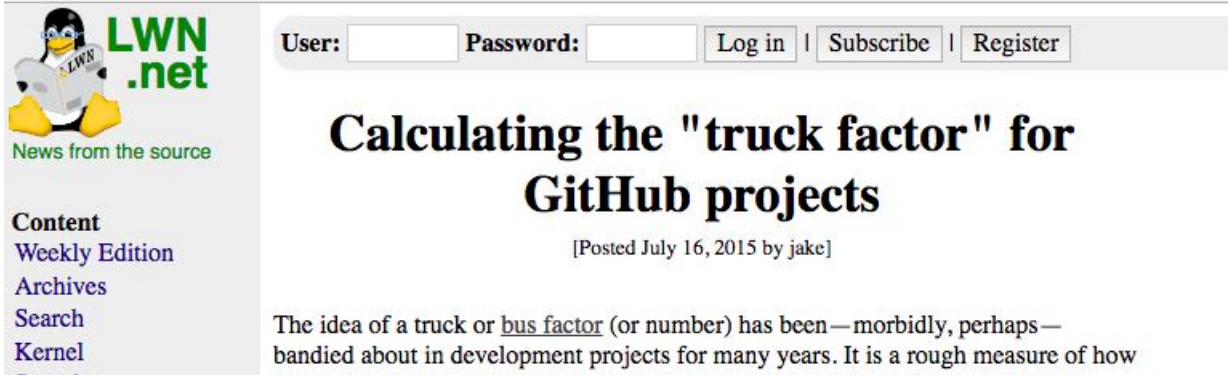
Department of Computer Science, UFMG, Brazil

{gaa,mtov,hora}@dcc.ufmg.br

Abstract

The Truck Factor designates the minimal number of developers that have to be hit by a truck (or quit) before a project is incapacitated. It can be seen as a measurement of the concentration of information in individual team members. We calculate the Truck Factor for 133 popular GitHub applications, in six languages. Results show that most systems have a small Truck Factor (34% have Truck Factor=1 and 30% have Truck Factor=2).

Practical Impact



"**This is a great idea.** It produced some **surprises**, like sprockets having a truck factor of 1."

"This is **great avenue for research** and showing important weaknesses in some projects with some **surprising results**."

Investigating Truck Factor Events



Research Questions

RQ1: How common are TF events?

RQ2: How often the projects survive a TF event?

RQ3: How surviving projects differ from non-surviving ones?

Dataset

- Top-500 in 6 languages
- Filtering step



1,932

Identifying Truck Factor Events



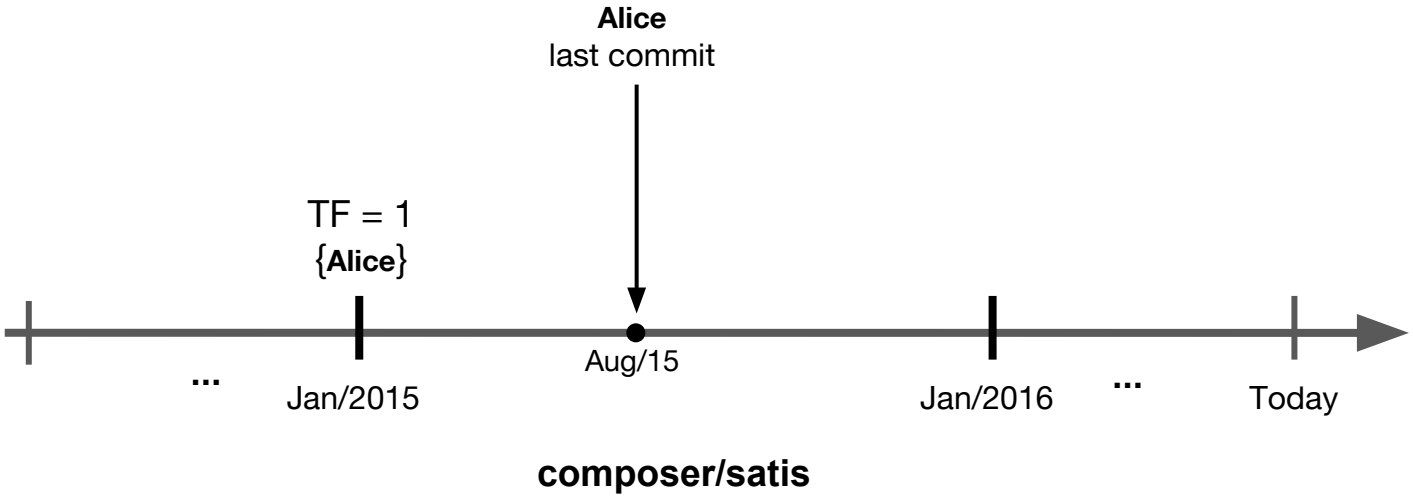
composer/satis

Identifying Truck Factor Events

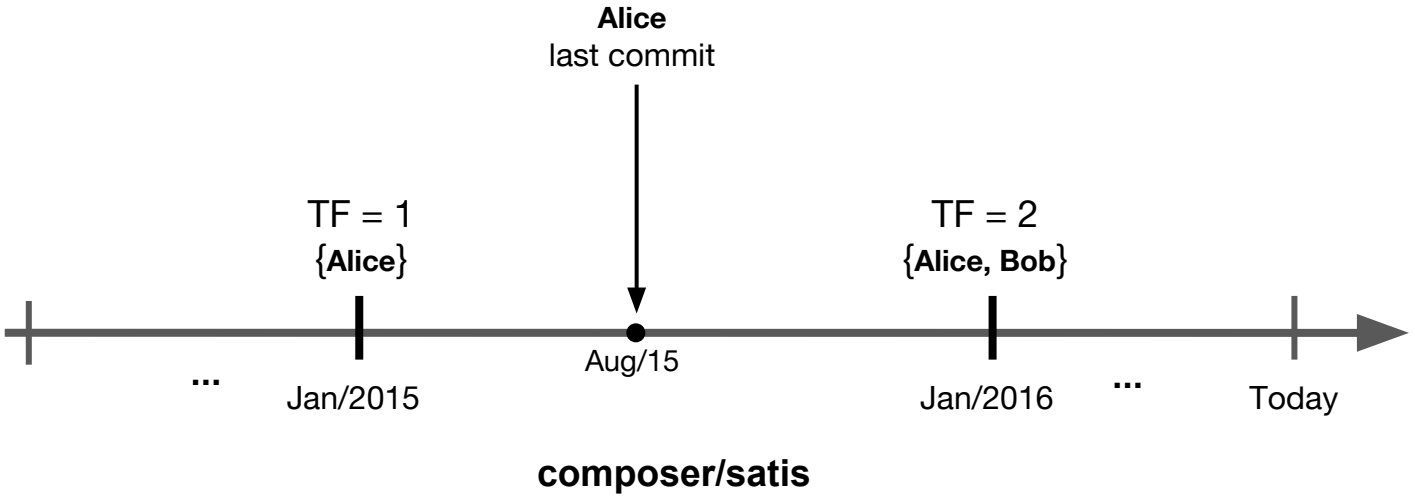


composer/satis

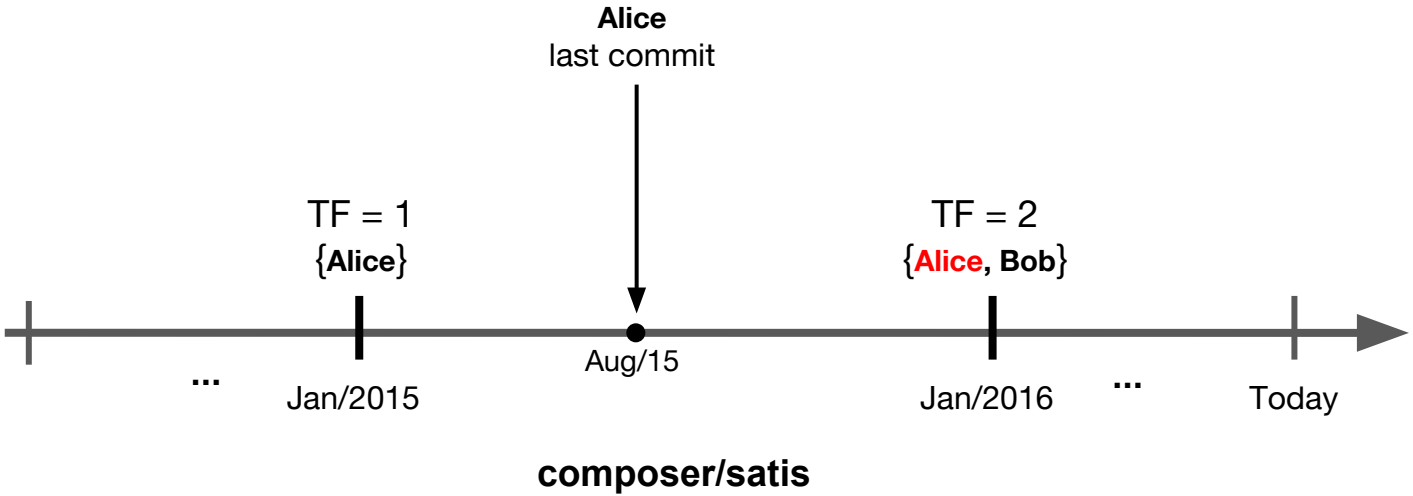
Identifying Truck Factor Events



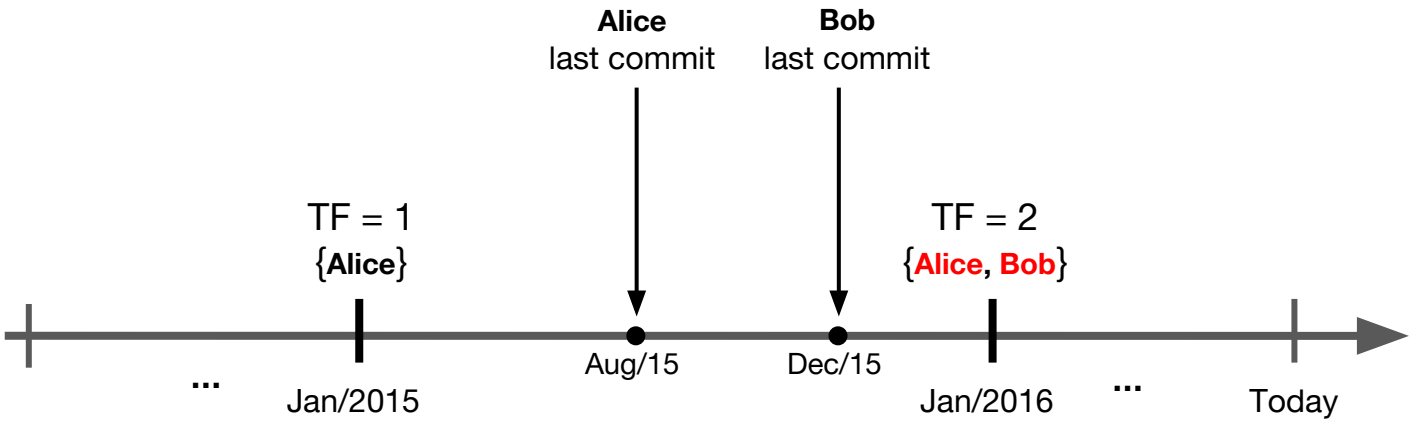
Identifying Truck Factor Events



Identifying Truck Factor Events

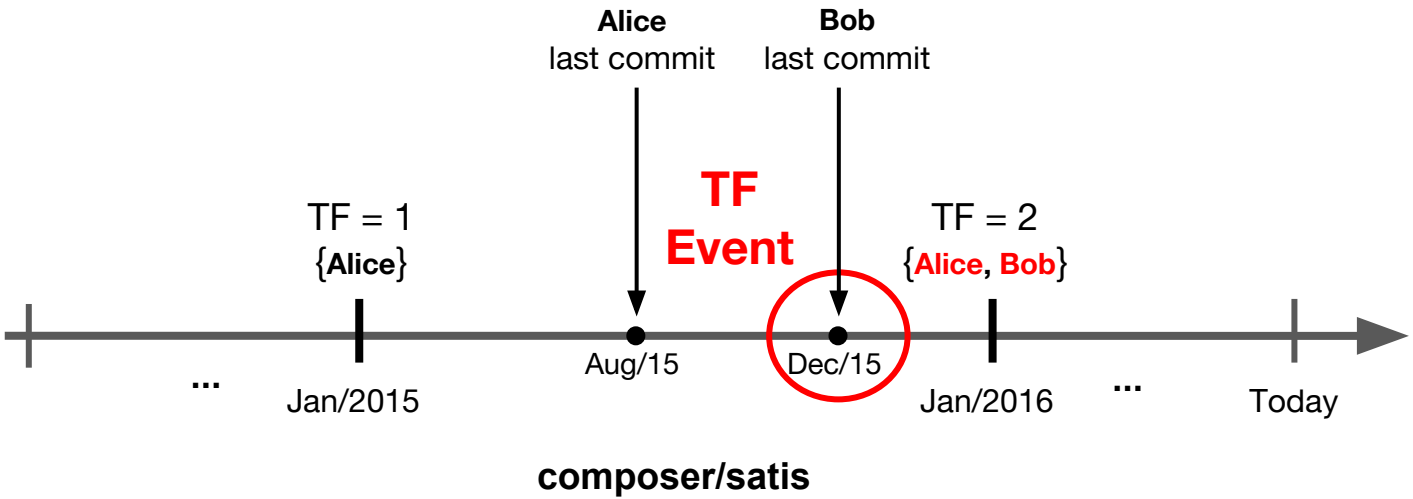


Identifying Truck Factor Events



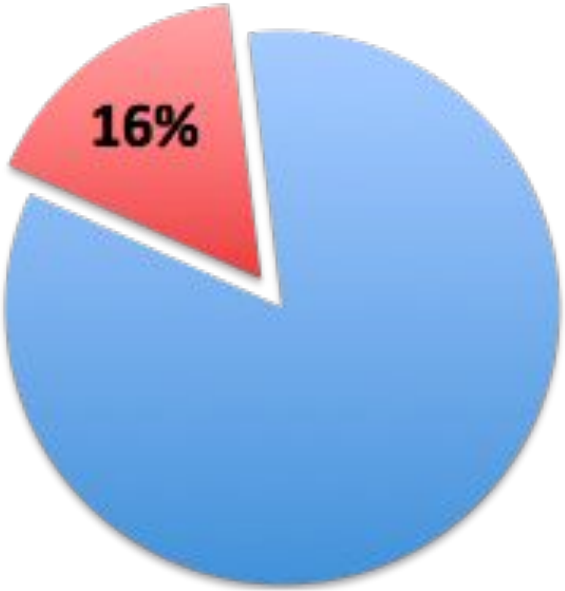
composer/satis

Identifying Truck Factor Events



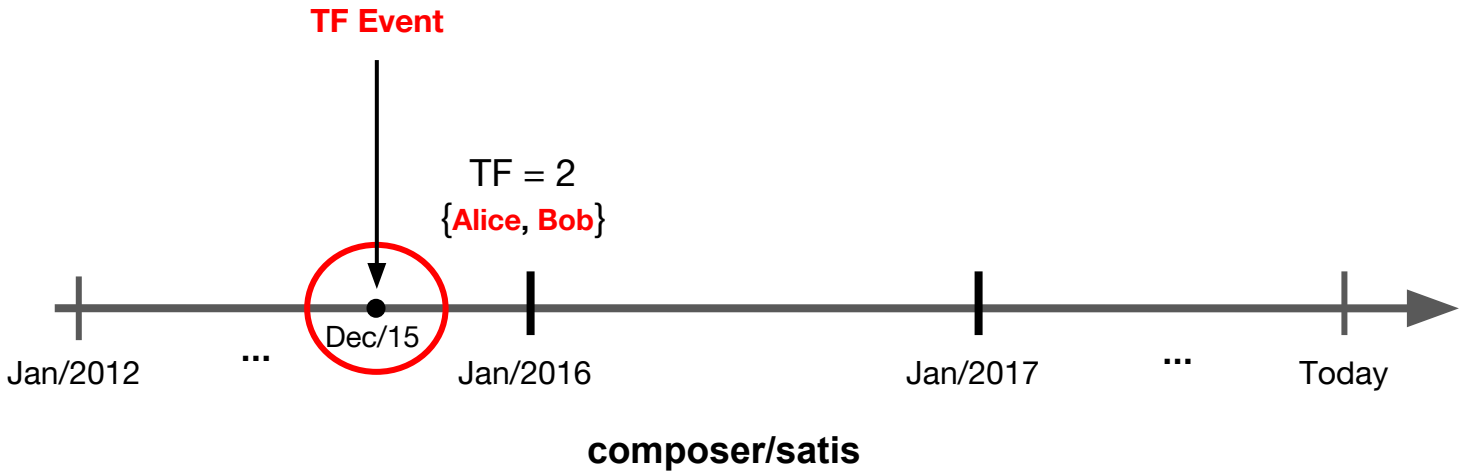
RQ1. Truck Factor Events

315 projects

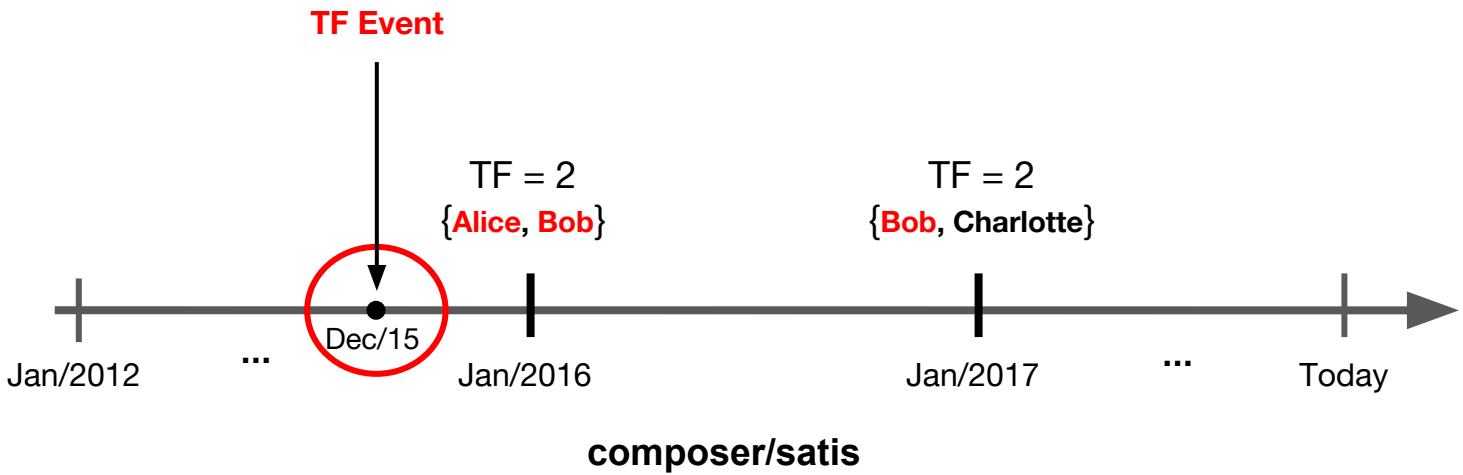


- TF event occur in 315 projects:
  - 66% => TF = 1
  - 59% => < 2 years of development

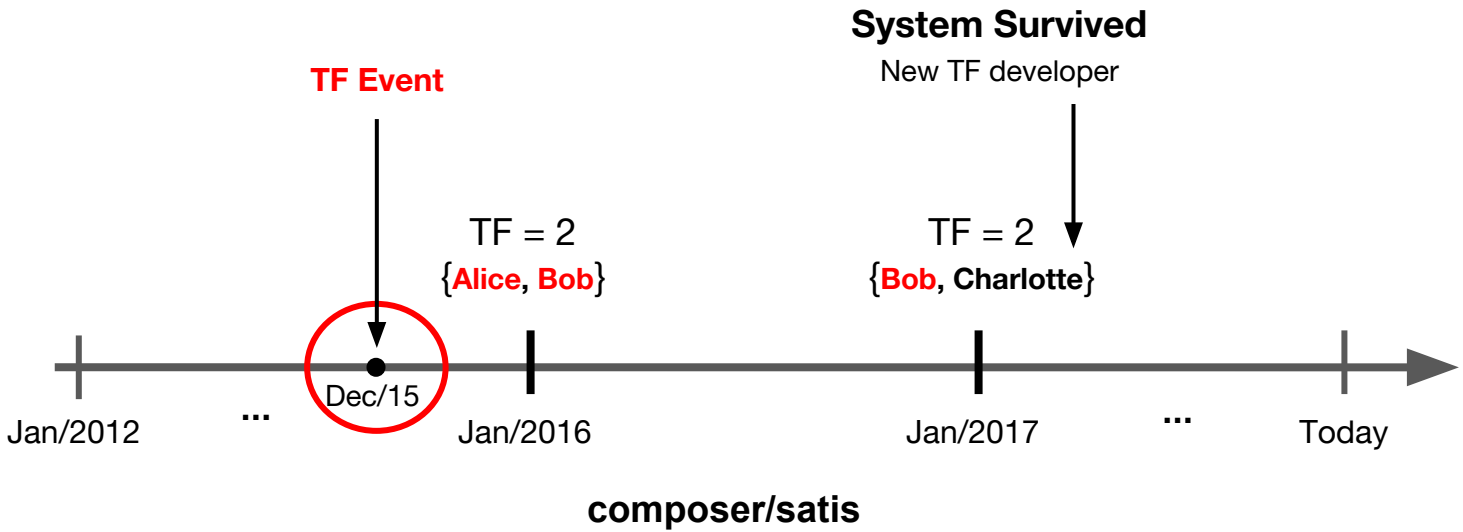
Identifying Surviving Projects



Identifying Surviving Projects

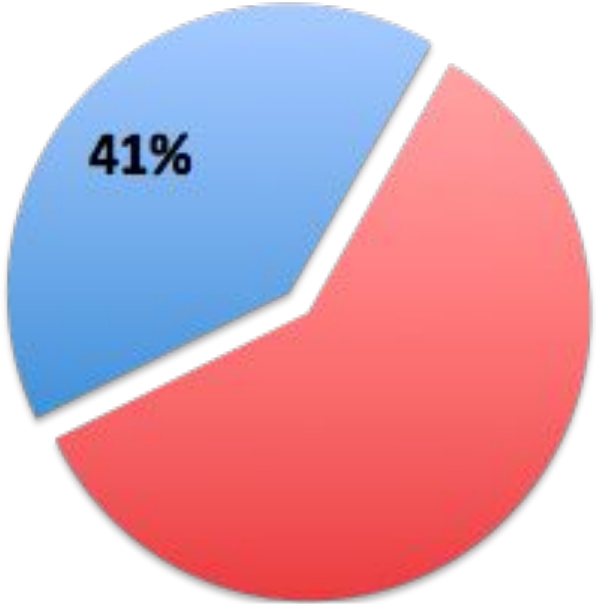


Identifying Surviving Projects



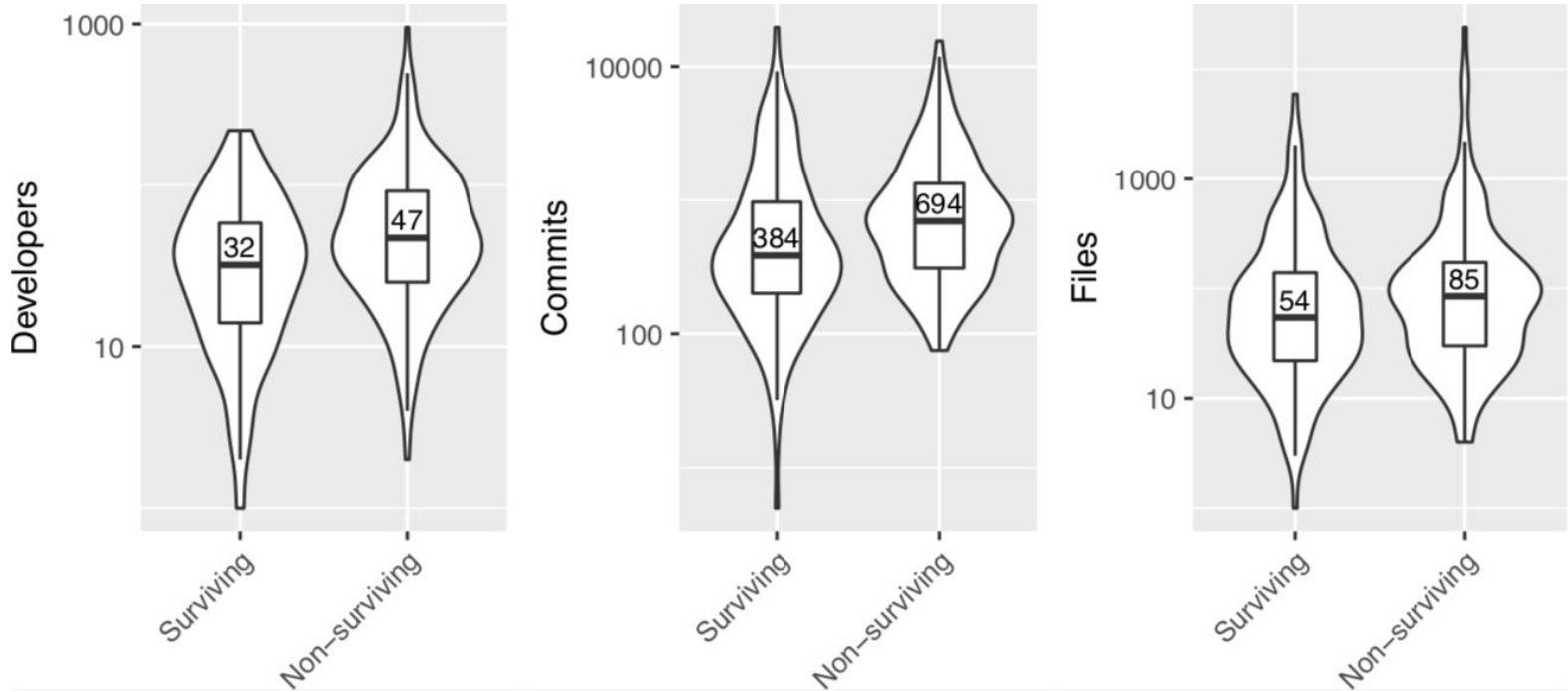
RQ2. Survival Rate

128 surviving projects

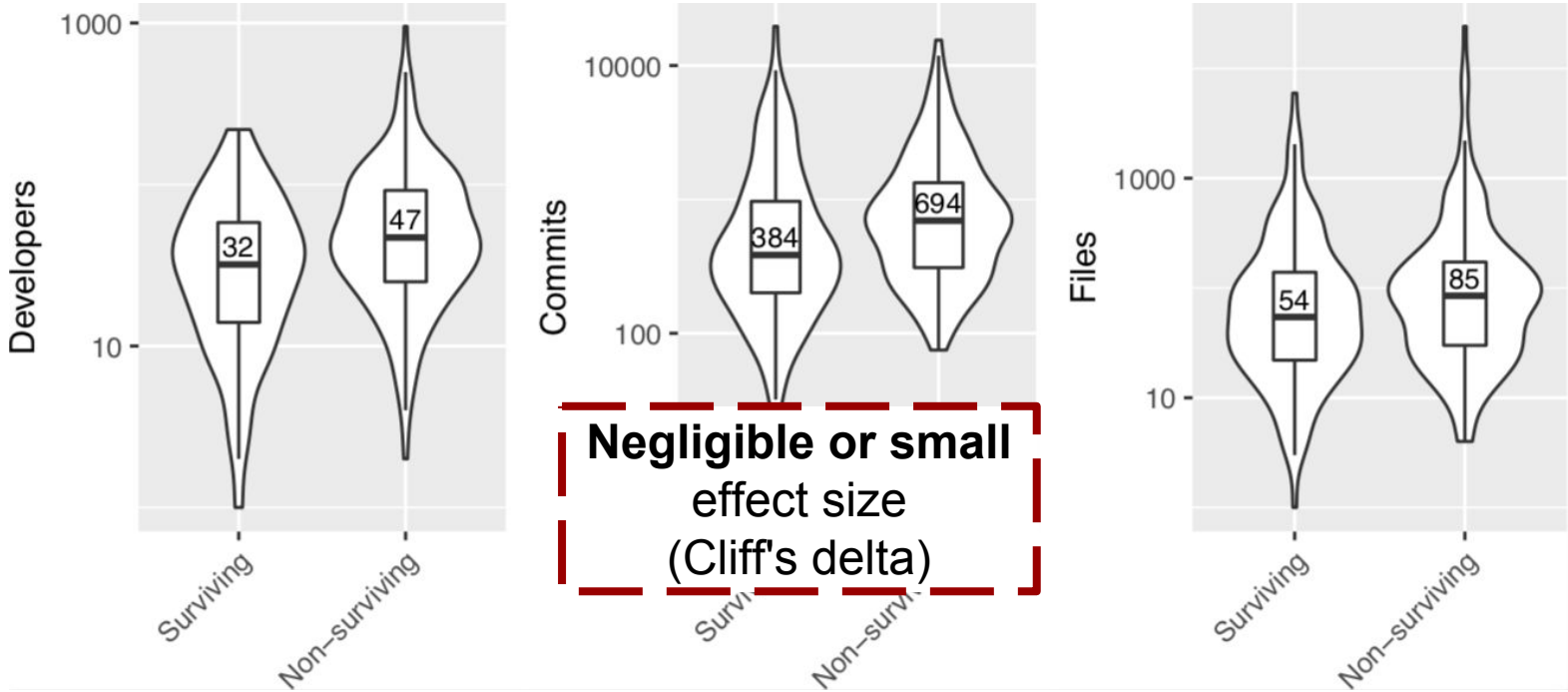


- 128 Surviving projects
  - 64% => one year after the event
  - 48% => helped by newcomers

RQ3. Surviving vs Non-surviving



RQ3. Surviving vs Non-surviving



Negligible or small effect size (Cliff's delta)

Survey

- Target: new TF developers
- 140 valid emails
- 33 answers (**response rate of 24%**)

Discontinuation Risks

80% agree/partially agree

Motivations

Motivations	Devs	%
Because I was using the project	17	53
To contribute to an open source project	11	34
To avoid the project discontinuation	5	16
I have interest on the project area	4	13
I get paid to contribute	4	13
To improve my own skills	3	9
I have the skills required by this project	3	9
It is a successful project	3	9
Others	7	22

Enablers and Barriers

	Enablers	Barriers
Human/Social	**Friendly and active owners/members**	**Lack of time**

"it has been [dev-name]'s **kindness** to my first contributions (...) and later other cool **developers' support**."

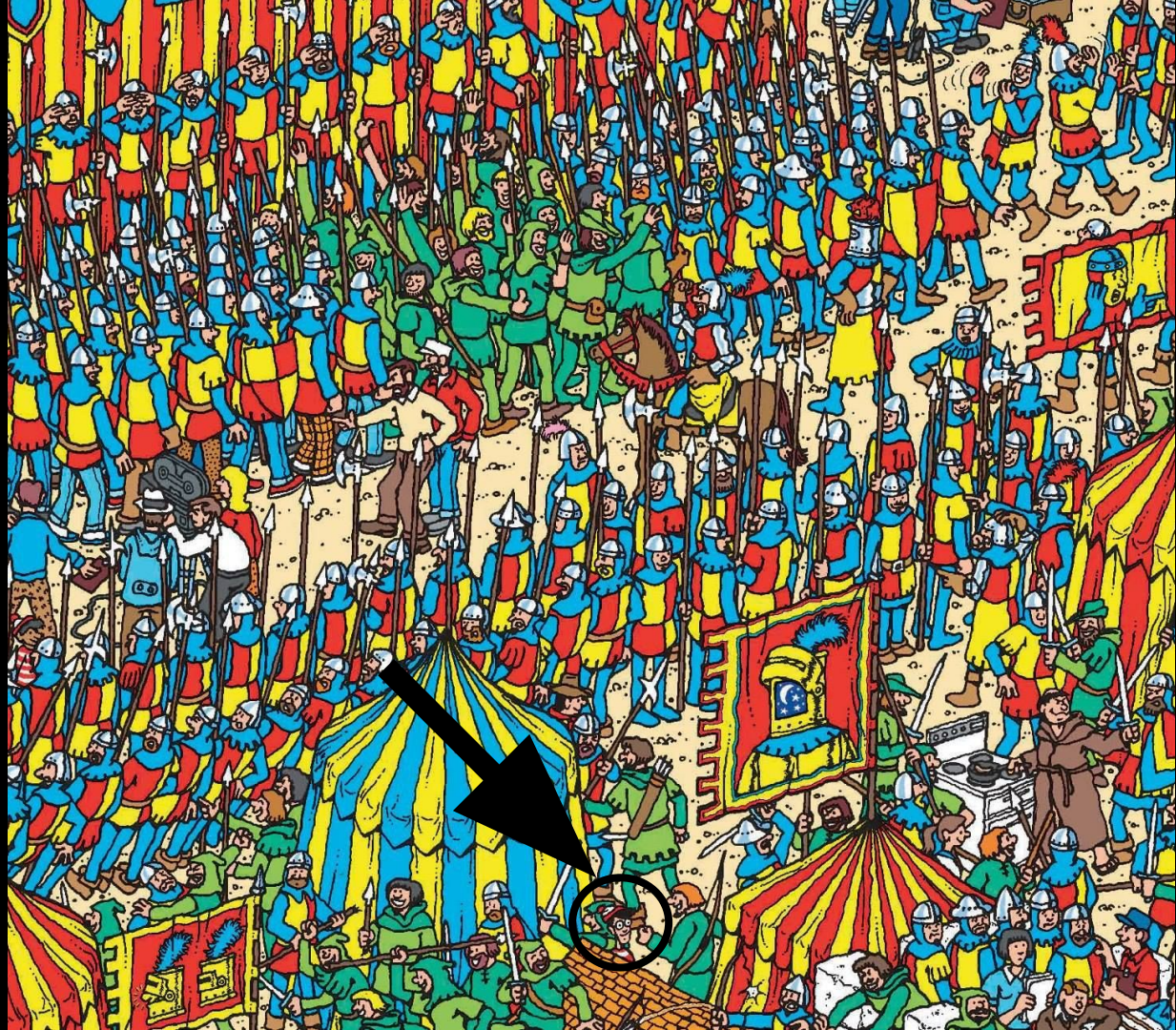
"**time** is always an issue"

Contributions

1. **Methodology** to identify **TF events/surviving systems**

2. **Findings** about **TF events/surviving systems**:
   - **TF events** are real (**16%** of the projects)
   - But **41%** of the projects **survive**
   - **Project usage** is the main motivation to contribute
   - **Human & social** aspects are key enablers/barriers

Identifying Software Maintainers



Techniques

- Number of commits (*Commits*)[1]
- Number of LOCs in the last version (*Blame*)[2]
- Degree of Authorship (*DOA*)[3]

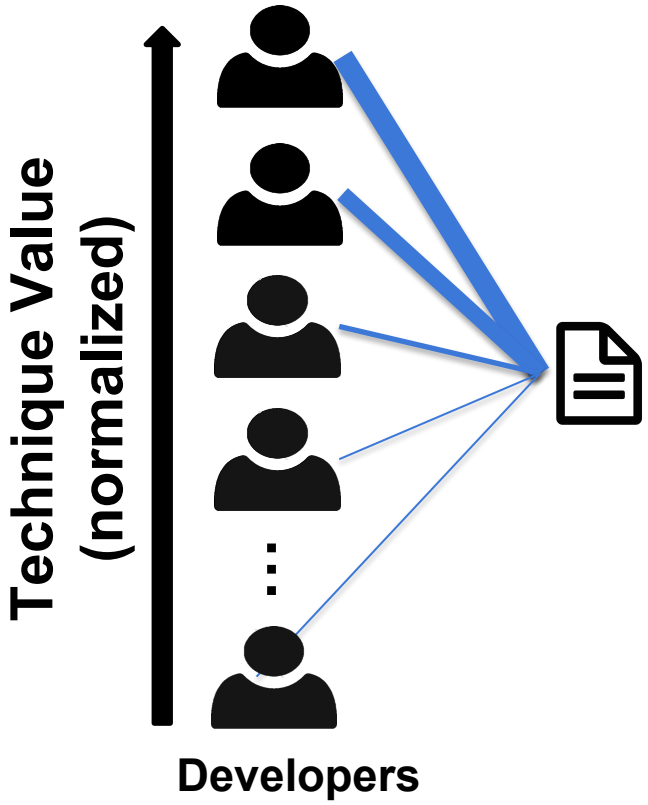
[1] Bird, C., et al. Don't touch my code! FSE 2011.

[2] Girba, T., et al. How developers drive software evolution. IWPSE 2005.

[3] Fritz, T., et al. Degree-of-knowledge: modeling a developer's knowledge of code. ACM TOSEM 2014.

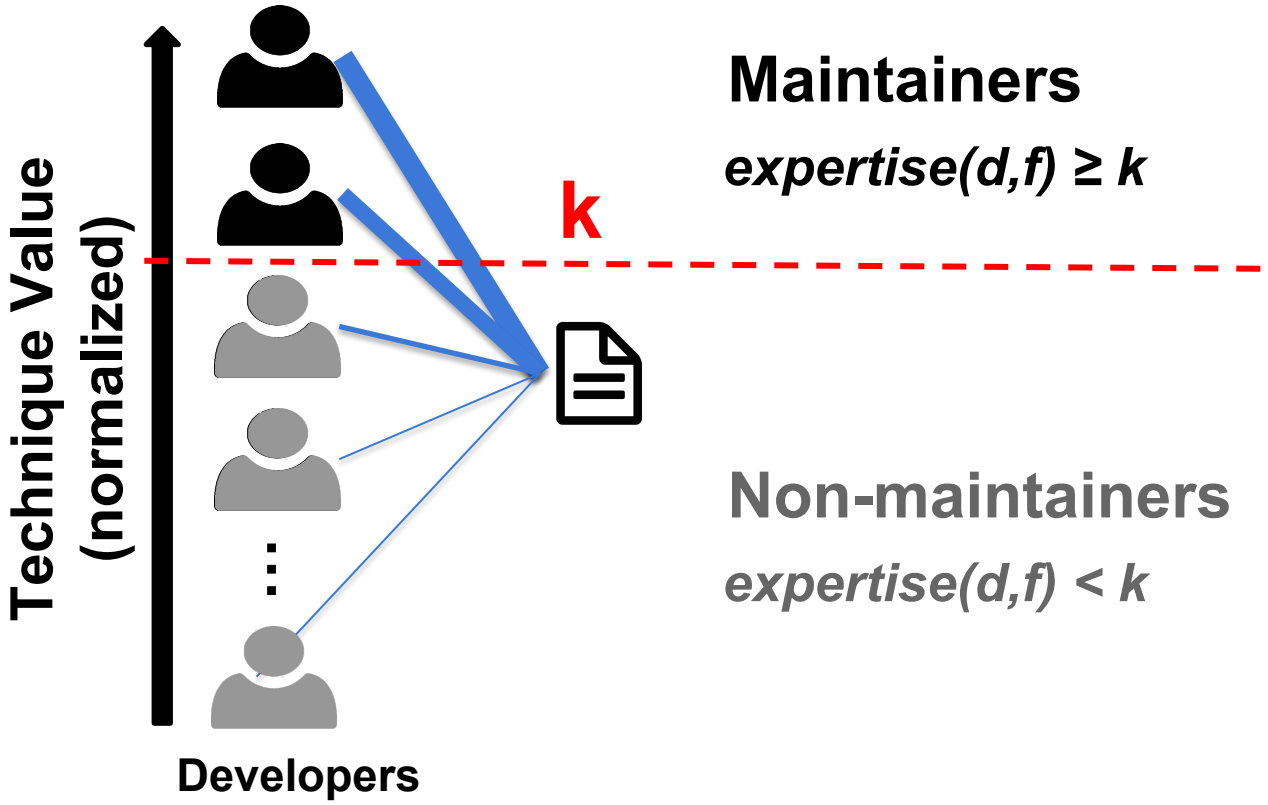
Inferring Maintainers and Non-maintainers

expertise(d,f) =

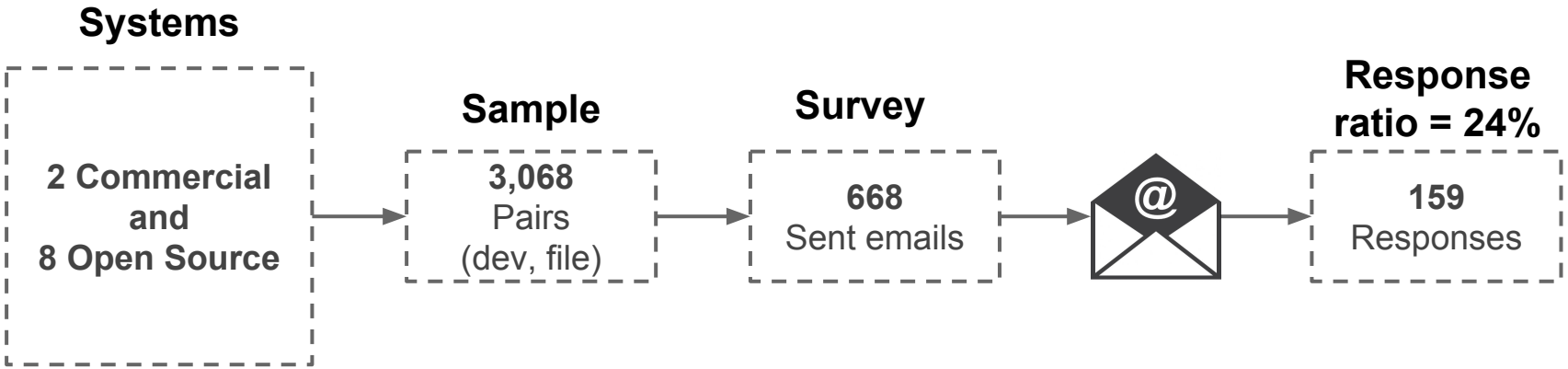


Inferring Maintainers and Non-maintainers

expertise(d,f) =



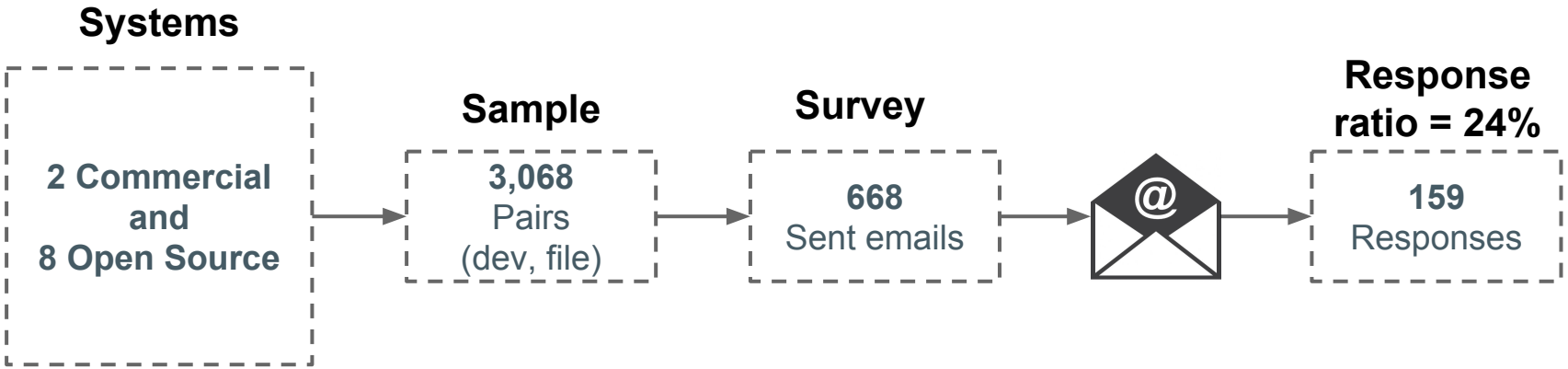
Oracle

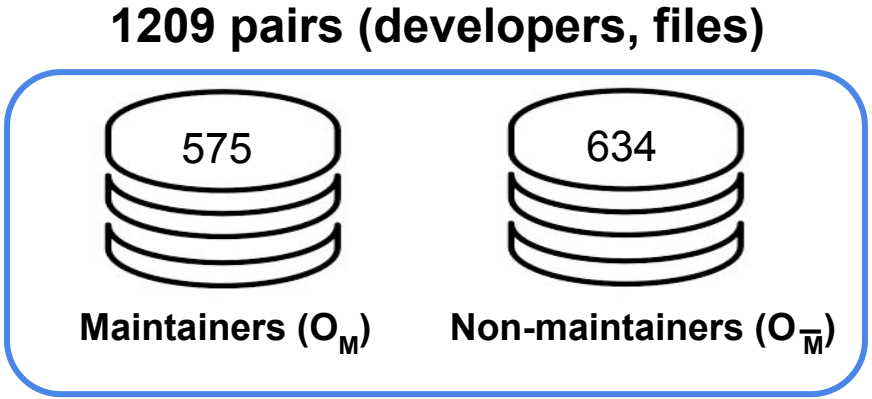


Please, access your knowledge about the source code in the following files.
Scale 1 to 5: (1) no knowledge; and (5) complete knowledge.

….

Oracle


Systems
2 Commercial and 8 Open Source
Sample
3,068 Pairs (dev, file)
Survey
668 Sent emails
@
Response ratio = 24%
159 Responses


1209 pairs (developers, files)
575
634
Maintainers (O_M)
Non-maintainers ($O_{\overline{M}}$)

Evaluation Metrics

Maintainers hit ratio (HR_M)

$$HR_M(k) = \frac{|\{(d,f) \in O_M \mid expertise(d,f) \geq k\}|}{|O_M|}$$

Non-maintainers hit ratio ($HR_{\overline{M}}$)

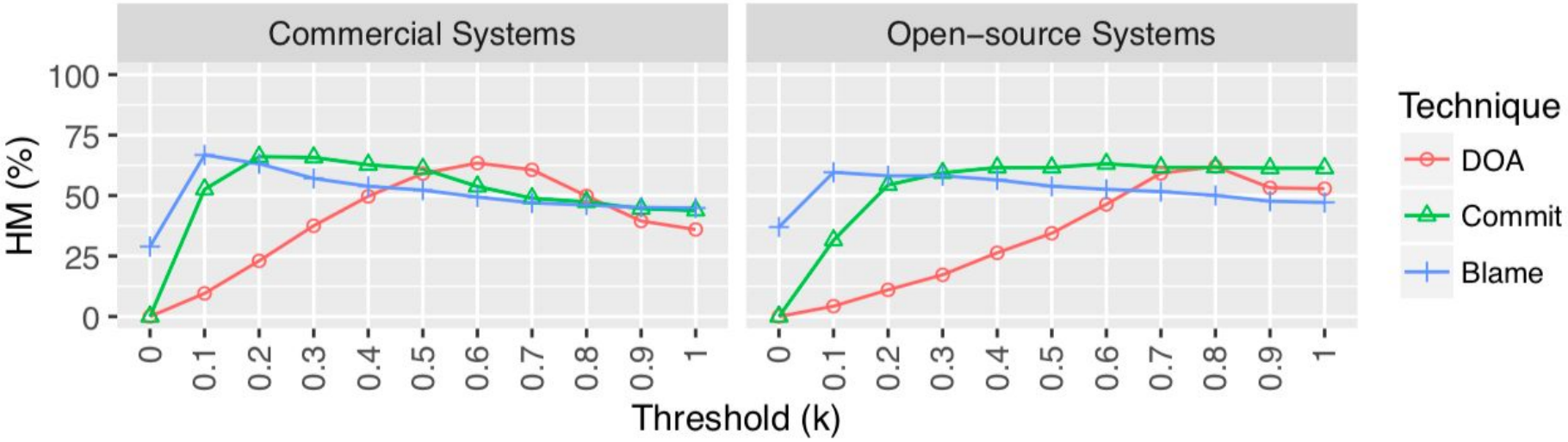
$$HR_{\overline{M}}(k) = \frac{|\{(d,f) \in O_{\overline{M}} \mid expertise(d,f) < k\}|}{|O_{\overline{M}}|}$$

Evaluation Metrics

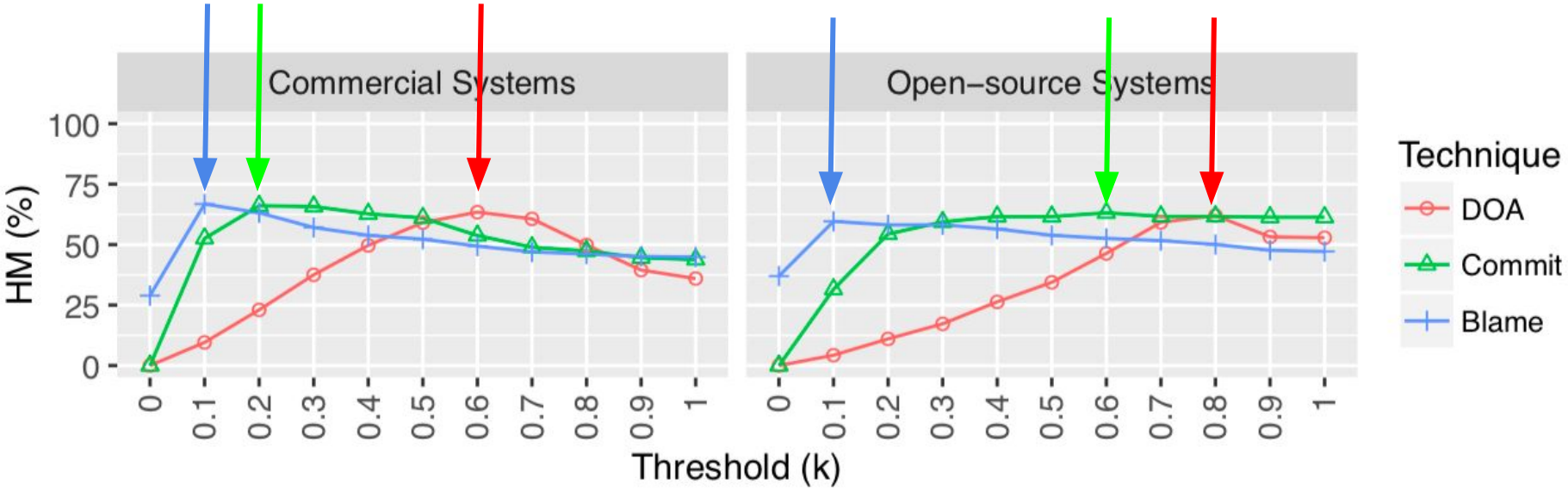
Harmonic Mean (HM)

$$HM(k) = \frac{2 * HR_M(k) * HR_{\overline{M}}(k)}{HR_M(k) + HR_{\overline{M}}(k)}$$

Results


Commercial Systems
Open−source Systems
HM (%)
100
75
50
25
0
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
Threshold (k)
Technique
DOA
Commit
Blame

Results



Commit, Blame, and DOA present similar performance (HM ranging from 60% to 67%)

Why the Techniques Fail

- AllFail = 96 pairs
  - **Recency**
    - ≤ 30 days (25%)
    - Last to change (11%)
  - File size
    - ≤ 26 LOC (14%)
  - Extra repository

"I don’t know if you are taking **time into account**, but I’d expect this to be a significant factor."

Why the Techniques Fail

- AllFail = 96 pairs
  - Recency
    - ≤ 30 days (25%)
    - Last to change (11%)
  - **File size**
    - ≤ 26 LOC (14%)
  - Extra repository

"**This is a tiny file**, so having added one line of code, I've contributed a significant portion."

Why the Techniques Fail

- AllFail = 96 pairs
  - Recency
    - ≤ 30 days (25%)
    - Last to change (11%)
  - File size
    - ≤ 26 LOC (14%)
  - **Extra repository**

"The knowledge came from **participating in the file definition**."

Controlling File Size and Recency

Maintainers

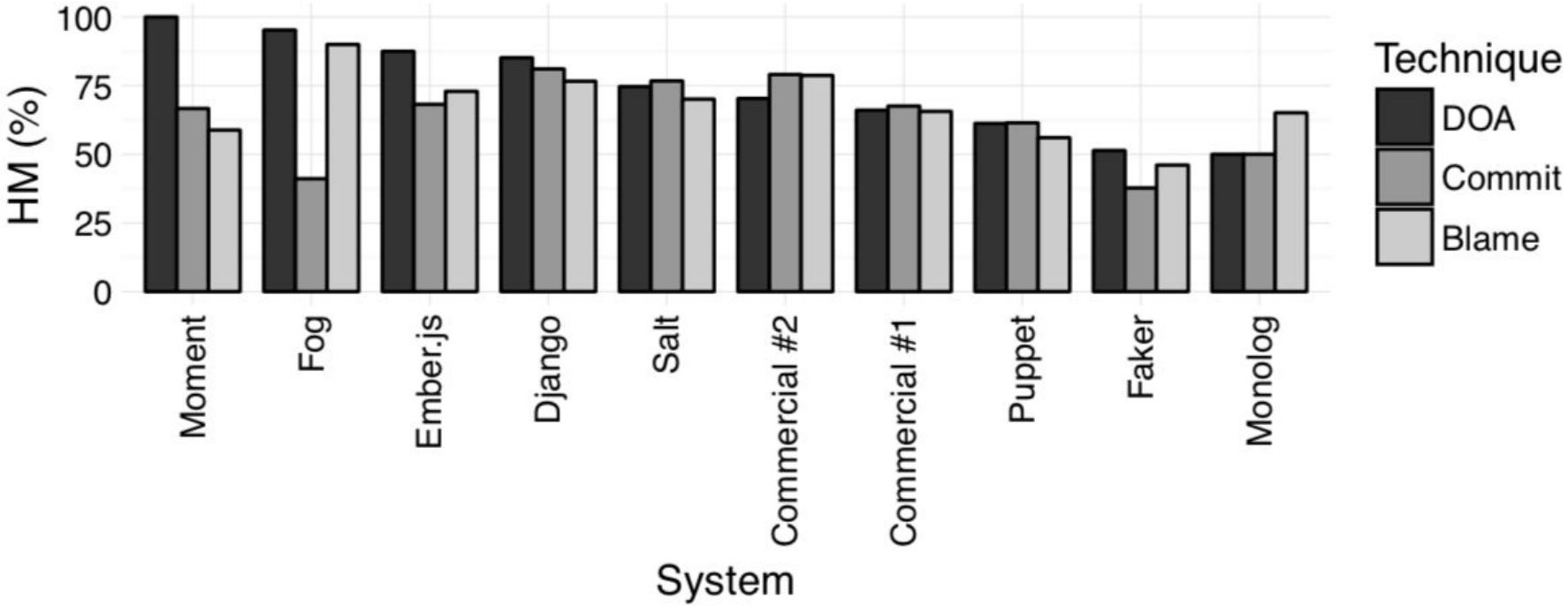
expertise(d,f) ≥ k

			
Small	**Large**	**Short**	**Long**
$k = k - k_1$	$k = k + k_2$	$k = k - k_3$	$k = k + k_4$

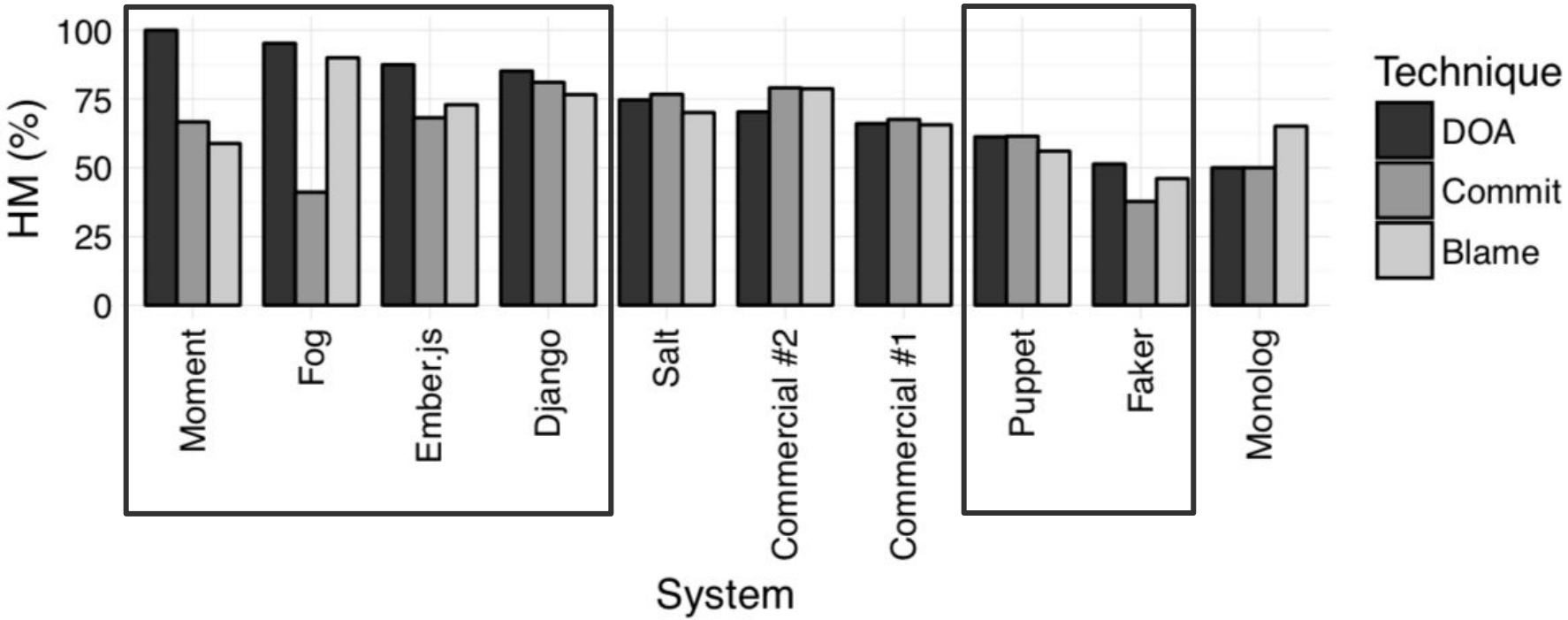
Improvements

- Blame: 14%
- Commit: 41%
- DOA: 95%

Harmonic Mean



Harmonic Mean



Contributions

1. Results on the **effectiveness** of code authorship techniques to suggest code maintainers
2. **Findings** about **techniques limitations** and recommendations on **how to improve them**

Conclusion

Conclusion

- We explored the **use of code authorship metrics** to understand the organization of **software teams** and to identify **key developers** (in projects and specific files)

- We **proposed new methodologies**, conducted **empirical investigations**, and **surveyed systems developers**

Contributions

1. A **methodology** and a set of **concepts** to study the organization of software teams
2. A novel **approach** to estimate truck factors
3. **Motivations, enablers, and barriers** that developers face when helping projects to overcome TF events
4. Results about the **effectiveness** of code authorship techniques on suggesting code maintainers

Future Work

- Investigate to which extent our results apply to other systems, in special commercial ones
- Introduce and evaluate improvements suggested by the surveyed developers

Publications

- Who Can Maintain this Code? Assessing the Effectiveness of Repository-Mining Techniques for Identifying Software Maintainers. **IEEE Software** 2018.
- Assessing Code Authorship: The Case of the Linux Kernel. **OSS** 2017.
- A Novel Approach for Estimating Truck Factors. **ICPC** 2016.
- Measuring and Analyzing Code Authorship in 1+114 Open Source Projects. [submitted to journal].

Identifying key developers in software projects using code authorship metrics

Thank you!

aSERG / UFMG
APPLIED SOFTWARE ENGINEERING RESEARCH GROUP

PPGCC
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO DCC-UFMG

UFPI
1971
UNIVERSIDADE FEDERAL DO PIAUÍ